Final Portfolio COMP 2040

Benny Chen

April 2026

Contents

Time to complete Portfolio: 12 hours

1 PS0: Hello SFML

1.1 Assignment Overview

Introduction to the foundations of the SFML Library. creating a window, loading sprites, and animating objects on screen. The assignment required sprites that move, react to keyboard input, and include something unique. I set up the build system using a Makefile, followed the SFML tutorial to render the initial window, and extended it with WASD movement and sprite collision.

1.2 Key Concepts and Design Choices

The central structure is the game loop where each frame polls sf::Events, and updates sprite positions based on held keys, then clears, draws, and calls display(). Collision detection uses axis-aligned bounding box checks by comparing the sf::FloatRect returned by getGlobalBounds() for each pair of sprites. I also set a framerate limit to keep motion smooth and frame-rate independent.

1.3 What I Learned

PS0 introduced me to the SFML event model and the immediate-mode render loop that is used in later assignments: clear, draw, display, set frame rate. I learned that floating-point positions that move by integer pixels can cause jitter at certain frame rates. After PS0, I was comfortable reading SFML documentation and reasoning about coordinate transforms, which were used in the later project.

1.4 Evidence

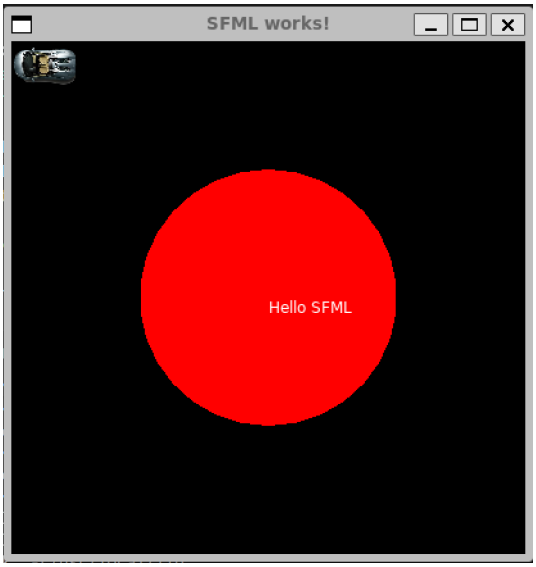


Figure 1: PS0 running: sprites moving with WASD input and border collision

1.5 Incomplete Features

The sprite collision is imperfect, it has a rectangular character model around circular sprites, so collisions trigger slightly early at the corners. A proper circle-to-circle distance check would fix this.

1.6 Source Code

Makefile

```
CC = g++
CFLAGS = --std=c++20 -Wall -Werror -pedantic -g
LIBPATH = -L/usr/local/lib
LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_unit_test_framework
# Your .hpp files
DEPS =
# Your compiled .o files
OBJECTS =
# The name of your program
PROGRAM = sfml-app

.PHONY: all clean lint


all: $(PROGRAM)

# Wildcard recipe to make .o files from corresponding .cpp file
%.o: %.cpp $(DEPS)
  $(CC) $(CFLAGS) -c $<

$(PROGRAM): main.o $(OBJECTS)
  $(CC) $(CFLAGS) -o $@ $^ $(LIB) $(OBJECTS) $(LIBPATH)

clean:
  rm *.o $(PROGRAM)

lint:
  cpplint *.cpp *.hpp
```

main.cpp

```
// Copyright 2026 Benny Chen
#include <SFML/Graphics.hpp>
int main() {
    // windows
    sf::RenderWindow window(sf::VideoMode({400, 400}), "SFML works!");
    sf::VideoMode desktop = sf::VideoMode::getDesktopMode();
    sf::Vector2u windowSize = window.getSize();
    // set to center of screen instead of bottom right
    window.setPosition({static_cast<int>((desktop.size.x - windowSize.x) / 2),
                        static_cast<int>((desktop.size.y - windowSize.y) / 2)});
    // shape
    sf::CircleShape shape(100.f);
    shape.setOrigin({100.f, 100.f});
    shape.setFillColor(sf::Color::Red);
    shape.setPosition({window.getSize().x / 2.f,
                       window.getSize().y / 2.f});
    // Create a graphical text to display
    const sf::Font font("arial.ttf");
    sf::Text text(font, "Hello SFML", 12);
    text.setPosition({window.getSize().x / 2.f,
                      window.getSize().y / 2.f});
    // Load a sprite to display
    const sf::Texture texture("sprite.png");
    sf::Sprite sprite(texture);
    sf::Vector2u texSize = texture.getSize();
    float scaleX = 52.f / texSize.x;
    float scaleY = 38.f / texSize.y;
    sprite.setScale(sf::Vector2f{scaleX, scaleY});
    ////////////////////////////////////////////////////////////////
    while (window.isOpen()) {
```

```
        window.setVerticalSyncEnabled(true);
        window.setFramerateLimit(240);
        while (const std::optional event = window.pollEvent()) {
            if (event->is<sf::Event::Closed>())
                window.close();
        }
        // movement speed
        sf::Vector2f movement(0.f, 0.f);
        float speed = 1.f;
        // movement
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::A)) {
            movement.x -= speed;
            sprite.setRotation(sf::degrees(180.f));
        }
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::D)) {
            movement.x += speed;
            sprite.setRotation(sf::degrees(0.f));
        }
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::W)) {
            movement.y -= speed;
            sprite.setRotation(sf::degrees(270.f));
        }
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::S)) {
            movement.y += speed;
            sprite.setRotation(sf::degrees(90.f));
        }
        sprite.move(movement);
        // close
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Key::Escape)) {
            window.close();
        }
        // display
        window.clear();
        window.draw(shape);
        window.draw(sprite);
        window.draw(text);
        window.display();
    }
}
```

2 PS1: Sprite Builder

2.1 Assignment Overview

PS1 had two parts. Part A required writing unit Boost tests before implementing the SpriteSheet class. Part B built a SpriteBuilder that randomly assembles character sprites from a tile sheet. The program is invoked as ./SpriteBuilder –file=class.txt –scale=4 –seed=12345. Pressing **R** rerolls a new random sprite, **S** saves the current sprite to a PNG, and **Esc** quits.

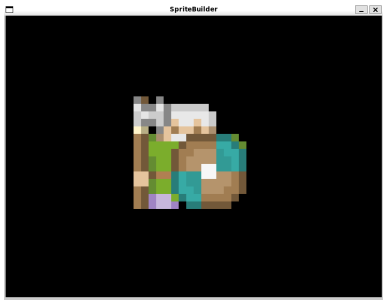
2.2 Key Concepts and Design Choices

SpriteSheet divides a large texture into a uniform grid based on tile size and optional margin. Given a row and column index, it computes the pixel rectangle with simple arithmetic and returns an sf::IntRect wrapper. The sf::Texture is stored in a shared_ptr so multiple objects can share the same texture without copying it. SpriteBuilder uses a Mersenne Twister (std::mt19937) seeded from the command-line argument to pick tile indices for each character layer (body, head, accessory, etc.), ensuring reproducible results for the same seed.

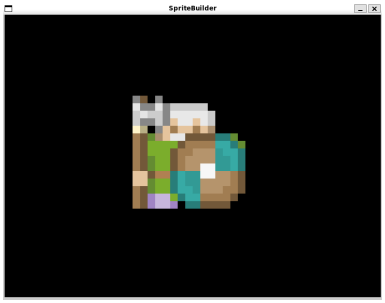
2.3 What I Learned

Test development was the biggest takeaway. Writing tests first forced me to decide the interface before the implementation: what getTile() should do for an out-of-bounds index? I chose to throw std::out_of_range, and locking that decision in early made the implementation cleaner and faster. I also learned how shared_ptr prevents redundant texture copies, a pattern I reused in later assignments.

2.4 Evidence



(a) PS1: A randomly assembled character sprite generated from the rogue tile sheet



(b) PS1: 'R' Used to retry

2.5 Incomplete Features

No known incomplete features. The margin arithmetic in SpriteSheet was fiddly—off-by-one errors in the pixel rectangle caused tiles to bleed into adjacent ones early on, but the unit tests caught these before submission.

2.6 Source Code

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Werror -pedantic -I/usr/local/include
SFML_FLAGS = -L/usr/local/lib -lsfml-graphics -lsfml-window -lsfml-system -Wl,-rpath,/usr/local/lib
BOOST_TEST_FLAGS = -lboost_unit_test_framework

```

```
SOURCES = SpriteSheet.cpp SpriteBuilder.cpp
OBJECTS = $(SOURCES:.cpp=.o)

MAIN_SRC = main.cpp
MAIN_OBJ = $(MAIN_SRC:.cpp=.o)

TEST_SRC = test.cpp
TEST_OBJ = $(TEST_SRC:.cpp=.o)

EXECUTABLE = SpriteBuilder
LIBRARY = SpriteBuilder.a
TEST_EXECUTABLE = test

all: $(EXECUTABLE) $(LIBRARY) $(TEST_EXECUTABLE)

$(EXECUTABLE): $(OBJECTS) $(MAIN_OBJ)
  $(CXX) $(CXXFLAGS) -o $@ $^ $(SFML_FLAGS)

$(LIBRARY): $(OBJECTS)
  ar rcs $@ $^

$(TEST_EXECUTABLE): $(OBJECTS) $(TEST_OBJ)
  $(CXX) $(CXXFLAGS) -o $@ $^ $(SFML_FLAGS) $(BOOST_TEST_FLAGS)

%.o: %.cpp
  $(CXX) $(CXXFLAGS) -c $< -o $@

SpriteSheet.o: SpriteSheet.cpp SpriteSheet.hpp
SpriteBuilder.o: SpriteBuilder.cpp SpriteBuilder.hpp SpriteSheet.hpp
main.o: main.cpp SpriteSheet.hpp SpriteBuilder.hpp
test.o: test.cpp SpriteSheet.hpp

clean:
  rm -f $(OBJECTS) $(MAIN_OBJ) $(TEST_OBJ) $(EXECUTABLE) $(LIBRARY) $(TEST_EXECUTABLE)

.PHONY: all clean
```

main.cpp

```
// Copyright 2026 Benny

#include <algorithm>
#include <iostream>
#include <iomanip>
#include <sstream>

#include <SFML/Graphics.hpp>

#include "SpriteBuilder.hpp"
#include "SpriteSheet.hpp"

static std::string paddedCount(int count) {
  std::ostringstream ss;
  ss << std::setw(2) << std::setfill('0') << count;
  return ss.str();
}

int main(int argc, char *argv[]) {
  std::string filename;
  std::random_device rd;
  uint32_t seed = rd();
  unsigned int scale = 1;
  bool hasSeed = false;

  for (int i = 1; i < argc; ++i) {
    std::string arg = argv[i];
```

```
    if (arg.rfind("--file=", 0) == 0) {
      filename = arg.substr(7);
    } else if (arg.rfind("--seed=", 0) == 0) {
      seed = static_cast<uint32_t>(std::stoul(arg.substr(7)));
      hasSeed = true;
    } else if (arg.rfind("--scale=", 0) == 0) {
      scale = static_cast<unsigned int>(std::stoul(arg.substr(8)));
    }
  }

  if (filename.empty()) {
    std::cerr << "Usage: " << argv[0]
              << " --file=<config.txt> [--seed=N] [--scale=N]" << std::endl;
    return 1;
  }

  std::string base = filename;
  size_t slash = base.find_last_of("/\\");
  if (slash != std::string::npos) base = base.substr(slash + 1);
  size_t dot = base.find_last_of('.');
  if (dot != std::string::npos) base = base.substr(0, dot);

  try {
    sb::SpriteBuilder builder = hasSeed
      ? sb::SpriteBuilder(filename, seed)
      : sb::SpriteBuilder(filename);

    builder.setScale(scale);

    sf::Vector2u winSize = builder.windowSize();
    unsigned int displayW = std::max(800u, winSize.x * scale);
    unsigned int displayH = std::max(600u, winSize.y * scale);
    sf::Vector2u displaySize(displayW, displayH);

    sf::RenderWindow window(
      sf::VideoMode(displaySize),
      "SpriteBuilder",
      sf::Style::Titlebar | sf::Style::Close);
    window.setFramerateLimit(60);

    int saveCount = 1;

    std::cout << "Press R to reroll. Press S to save. Press ESC to quit."
              << std::endl;

    while (window.isOpen()) {
      while (const std::optional event = window.pollEvent()) {
        if (event->is<sf::Event::Closed>()) {
          window.close();
        }
        if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>()) {
          if (keyPressed->code == sf::Keyboard::Key::Escape) {
            window.close();
          }
          if (keyPressed->code == sf::Keyboard::Key::R) {
            builder.randomize();
          }
          if (keyPressed->code == sf::Keyboard::Key::S) {
            sf::RenderTexture rt;
            if (rt.resize(winSize)) {
              rt.clear(sf::Color::Transparent);
              builder.setScale(1);
              rt.draw(builder);
              rt.display();
              builder.setScale(scale);

              std::string outFile = base + "_" + paddedCount(saveCount++) + ".png";
              bool saved = rt.getTexture().copyToImage().saveToFile(outFile);
```

```
            if (saved)
              std::cout << "Saved: " << outFile << std::endl;
            else
              std::cerr << "Failed to save: " << outFile << std::endl;
          }
        }
      }
    }

    window.clear(sf::Color::Black);
    window.draw(builder);
    window.display();
  }
 } catch (const std::exception &e) {
   std::cerr << "Error: " << e.what() << std::endl;
   return 1;
 }

 return 0;
}
```

Spritesheet.hpp

```
// Copyright 2026 Benny
#pragma once

#include <memory>
#include <string>

#include <SFML/Graphics.hpp>

namespace sb {

class TextureView {
 public:
  TextureView(std::shared_ptr<sf::Texture> texture, sf::IntRect region);

  TextureView crop(sf::IntRect area) const;

  sf::Vector2u getSize() const;
  sf::Sprite toSprite() const;
  sf::Image toImage() const;

 private:
  std::shared_ptr<sf::Texture> texture_;
  sf::IntRect region_;
};

class SpriteSheet {
 public:
  SpriteSheet(sf::Image img, sf::Vector2u tileSize, int margin = 0);
  SpriteSheet(const std::string &imgFilename, sf::Vector2u tileSize,
              int margin = 0);

  sf::Vector2u getTileSize() const;
  sf::Vector2u size() const;

  size_t length() const;
  size_t width() const;
  size_t height() const;

  TextureView operator[](sf::Vector2u pt) const;
  TextureView operator[](size_t i) const;

 private:
  std::shared_ptr<sf::Texture> texture_;
```

```
  sf::Vector2u tileSize_;
  int margin_;
  size_t gridWidth_;
  size_t gridHeight_;
};

}
// namespace sb
```

Spritesheet.cpp

```
// Copyright 2026 Benny

#include "SpriteSheet.hpp"
#include <stdexcept>

namespace sb {

// texture view
TextureView::TextureView(std::shared_ptr<sf::Texture> texture,
                         sf::IntRect region)
    : texture_(texture), region_(region) {
  if (!texture_) {
    throw std::invalid_argument("Texture cannot be null");
  }

  // check region in bound
  sf::Vector2u texSize = texture_->getSize();
  if (region_.position.x < 0 || region_.position.y < 0 ||
      region_.size.x < 0 || region_.size.y < 0 ||
      static_cast<unsigned>(region_.position.x + region_.size.x) > texSize.x ||
      static_cast<unsigned>(region_.position.y + region_.size.y) > texSize.y) {
    throw std::out_of_range("Region extends outside texture bounds");
  }
}

// crop area
TextureView TextureView::crop(sf::IntRect area) const {
  if (area.position.x < 0 || area.position.y < 0 || area.size.x < 0 ||
      area.size.y < 0 || area.position.x + area.size.x > region_.size.x ||
      area.position.y + area.size.y > region_.size.y) {
    throw std::out_of_range("Crop area extends outside view bounds");
  }

  sf::IntRect absoluteRegion(sf::Vector2i(region_.position.x + area.position.x,
                                          region_.position.y + area.position.y),
                             area.size);

  return TextureView(texture_, absoluteRegion);
}

sf::Vector2u TextureView::getSize() const {
  return sf::Vector2u(static_cast<unsigned>(region_.size.x),
                      static_cast<unsigned>(region_.size.y));
}

sf::Sprite TextureView::toSprite() const {
  sf::Sprite sprite(*texture_);
  sprite.setTextureRect(region_);
  return sprite;
}

// create image of texture
sf::Image TextureView::toImage() const {
  sf::Image fullImage = texture_->copyToImage();
  sf::Image result;
```

```
  result.resize(getSize());

  for (unsigned int y = 0; y < getSize().y; ++y) {
    for (unsigned int x = 0; x < getSize().x; ++x) {
      sf::Color pixel = fullImage.getPixel(
          sf::Vector2u(static_cast<unsigned>(region_.position.x) + x,
                       static_cast<unsigned>(region_.position.y) + y));
      result.setPixel(sf::Vector2u(x, y), pixel);
    }
  }

  return result;
}

// sprite sheet
SpriteSheet::SpriteSheet(sf::Image img, sf::Vector2u tileSize, int margin)
    : tileSize_(tileSize), margin_(margin) {
  if (tileSize.x == 0 || tileSize.y == 0) {
    throw std::invalid_argument("Tile size must be greater than 0");
  }

  if (margin < 0) {
    throw std::invalid_argument("Margin cannot be negative");
  }

  texture_ = std::make_shared<sf::Texture>();
  if (!texture_->loadFromImage(img)) {
    throw std::runtime_error("Failed to load texture from image");
  }

  sf::Vector2u imageSize = img.getSize();

  if (imageSize.x < static_cast<unsigned>(margin) + tileSize.x) {
    gridWidth_ = 0;
  } else {
    gridWidth_ = (imageSize.x + margin) / (tileSize.x + margin);
  }

  if (imageSize.y < static_cast<unsigned>(margin) + tileSize.y) {
    gridHeight_ = 0;
  } else {
    gridHeight_ = (imageSize.y + margin) / (tileSize.y + margin);
  }

  if (gridWidth_ == 0 || gridHeight_ == 0) {
    throw std::invalid_argument(
        "Image too small for given tile size and margin");
  }
}

SpriteSheet::SpriteSheet(const std::string &imgFilename, sf::Vector2u tileSize,
                         int margin)
    : tileSize_(tileSize), margin_(margin) {
  sf::Image img;
  if (!img.loadFromFile(imgFilename)) {
    throw std::runtime_error("Failed to load image from file: " + imgFilename);
  }

  if (tileSize.x == 0 || tileSize.y == 0) {
    throw std::invalid_argument("Tile size must be greater than 0");
  }

  if (margin < 0) {
    throw std::invalid_argument("Margin cannot be negative");
  }

  texture_ = std::make_shared<sf::Texture>();
  if (!texture_->loadFromImage(img)) {
```

```
    throw std::runtime_error("Failed to load texture from image");
  }

  sf::Vector2u imageSize = img.getSize();

  if (imageSize.x < static_cast<unsigned>(margin) + tileSize.x) {
    gridWidth_ = 0;
  } else {
    gridWidth_ = (imageSize.x - margin) / (tileSize.x + margin);
  }

  if (imageSize.y < static_cast<unsigned>(margin) + tileSize.y) {
    gridHeight_ = 0;
  } else {
    gridHeight_ = (imageSize.y - margin) / (tileSize.y + margin);
  }

  if (gridWidth_ == 0 || gridHeight_ == 0) {
    throw std::invalid_argument(
        "Image too small for given tile size and margin");
  }
}

sf::Vector2u SpriteSheet::getTileSize() const { return tileSize_; }

size_t SpriteSheet::width() const { return gridWidth_; }

size_t SpriteSheet::height() const { return gridHeight_; }

sf::Vector2u SpriteSheet::size() const {
  return sf::Vector2u(static_cast<unsigned>(gridWidth_),
                      static_cast<unsigned>(gridHeight_));
}

size_t SpriteSheet::length() const { return gridWidth_ * gridHeight_; }

TextureView SpriteSheet::operator[](sf::Vector2u pt) const {
  if (pt.x >= gridWidth_ || pt.y >= gridHeight_) {
    throw std::out_of_range("Grid position out of bounds");
  }

  int pixelX = margin_ + static_cast<int>(pt.x) *
                         (static_cast<int>(tileSize_.x) + margin_);
  int pixelY = margin_ + static_cast<int>(pt.y) *
                         (static_cast<int>(tileSize_.y) + margin_);
  int tileWidth = static_cast<int>(tileSize_.x);
  int tileHeight = static_cast<int>(tileSize_.y);

  return TextureView(texture_,
                     sf::IntRect(sf::Vector2i(pixelX, pixelY),
                                 sf::Vector2i(tileWidth, tileHeight)));
}

TextureView SpriteSheet::operator[](size_t i) const {
  if (i >= length()) {
    throw std::out_of_range("Index out of bounds");
  }

  unsigned int x = i % gridWidth_;
  unsigned int y = i / gridWidth_;

  return (*this)[sf::Vector2u(x, y)];
}

} // namespace sb
```

SpriteBuilder.hpp

```hpp
// Copyright 2026 Benny
#pragma once

#include <random>
#include <string>
#include <vector>

#include "SpriteSheet.hpp"
#include <SFML/Graphics.hpp>

namespace sb {

class SpriteBuilder : public sf::Drawable {
 public:
  explicit SpriteBuilder(const std::string &filename);
  SpriteBuilder(const std::string &filename, uint32_t seed);

  sf::Vector2u windowSize() const;

  void randomize();
  void setScale(unsigned int scale);

 protected:
  void draw(sf::RenderTarget &window, sf::RenderStates states) const override;

 private:
  std::mt19937 rng_;
  sf::Vector2u tileSize_;
  int margin_;
  sf::Vector2u windowSize_;
  unsigned int scale_ = 1;

  std::vector<SpriteSheet> layers_;
  std::vector<size_t> currentSelection_;
};

} // namespace sb
```

SpriteBuilder.cpp

```cpp
// Copyright 2026 Benny

#include "SpriteBuilder.hpp"

#include <fstream>
#include <sstream>
#include <stdexcept>

namespace sb {

SpriteBuilder::SpriteBuilder(const std::string &filename)
    : SpriteBuilder(filename, std::random_device {}()) {}

SpriteBuilder::SpriteBuilder(const std::string &filename, uint32_t seed)
    : rng_(seed) {
  std::ifstream file(filename);
  if (!file.is_open()) {
    throw std::runtime_error("Failed to open configuration file: " + filename);
  }

  unsigned int tileWidth, tileHeight;
  file >> tileWidth >> tileHeight >> margin_;
  tileSize_ = sf::Vector2u(tileWidth, tileHeight);
```

```
  windowSize_ = tileSize_;

  std::string line;
  std::getline(file, line); // consume rest of first line

  while (std::getline(file, line)) {
    if (line.empty() ||
        line.find_first_not_of(" \t\n\r") == std::string::npos) {
      continue;
    }

    try {
      SpriteSheet sheet(line, tileSize_, margin_);
      layers_.push_back(sheet);
      currentSelection_.push_back(0);
    } catch (const std::exception &) {
      // skip unloadable sheets
    }
  }

  if (layers_.empty()) {
    throw std::runtime_error("No valid sprite sheets loaded");
  }

  randomize();
}

sf::Vector2u SpriteBuilder::windowSize() const { return windowSize_; }

void SpriteBuilder::setScale(unsigned int scale) {
  scale_ = (scale < 1) ? 1 : scale;
}

void SpriteBuilder::randomize() {
  for (size_t i = 0; i < layers_.size(); ++i) {
    std::uniform_int_distribution<size_t> dist(0, layers_[i].length() - 1);
    currentSelection_[i] = dist(rng_);
  }
}

void SpriteBuilder::draw(sf::RenderTarget &window,
                         sf::RenderStates states) const {
  sf::Vector2u targetSize = window.getSize();
  float scaledW = static_cast<float>(tileSize_.x) * static_cast<float>(scale_);
  float scaledH = static_cast<float>(tileSize_.y) * static_cast<float>(scale_);
  float offsetX = (static_cast<float>(targetSize.x) - scaledW) / 2.f;
  float offsetY = (static_cast<float>(targetSize.y) - scaledH) / 2.f;

  for (size_t i = 0; i < layers_.size(); ++i) {
    TextureView view = layers_[i][currentSelection_[i]];
    sf::Sprite sprite = view.toSprite();
    sprite.setScale(sf::Vector2f(static_cast<float>(scale_),
                                 static_cast<float>(scale_)));
    sprite.setPosition(sf::Vector2f(offsetX, offsetY));
    window.draw(sprite, states);
  }
}

} // namespace sb
```

test.hpp

```
// Copyright 2026 Benny Chen
#define BOOST_TEST_DYN_LINK
#define BOOST_TEST_MODULE SpriteSheet Tests

```

```cpp
#include <cstdio>
#include <fstream>
#include <iostream>
#include <random>
#include <set>
#include <string>

#include <boost/test/unit_test.hpp>

#include "SpriteSheet.hpp"
#include "SpriteBuilder.hpp"

using sb::SpriteSheet;
using sb::TextureView;
using sf::Image;
using sf::Vector2u;

namespace sf {
std::ostream &operator<<(std::ostream &os, const sf::Vector2u &v) {
  os << "<" << v.x << "," << v.y << ">";
  return os;
}
} // namespace sf

// Wrong dimensions test
BOOST_AUTO_TEST_CASE(TestWrongDimensions) {
  Image img("images/base.png");
  SpriteSheet sheet(img, {31, 32}, 1);
  BOOST_CHECK_EQUAL(sheet.width(), 1);
  BOOST_CHECK_EQUAL(sheet.height(), 2);
  BOOST_CHECK_EQUAL(sheet.length(), 2);
}

// Fixed position test
BOOST_AUTO_TEST_CASE(TestFixedPosition) {
  Image img("images/base.png");
  SpriteSheet sheet(img, {31, 32}, 1);

  TextureView tile0 = sheet[0];
  TextureView tile1 = sheet[1];
  sf::IntRect rect0 = tile0.toSprite().getTextureRect();
  sf::IntRect rect1 = tile1.toSprite().getTextureRect();
  BOOST_CHECK(rect0 != rect1);
}

// Column Test
BOOST_AUTO_TEST_CASE(TestColumn) {
  Image img("images/base.png");
  SpriteSheet sheet(img, {31, 32}, 1);

  TextureView tile0 = sheet[0];
  TextureView tile1 = sheet[1];

  BOOST_CHECK_EQUAL(tile0.getSize(), Vector2u(31, 32));
  BOOST_CHECK_EQUAL(tile1.getSize(), Vector2u(31, 32));
}

// Bad crop test
BOOST_AUTO_TEST_CASE(TestBadCrop) {
  Image img("images/base.png");
  SpriteSheet sheet(img, {31, 32}, 1);
  TextureView view = sheet[0];

  BOOST_CHECK_THROW(view.crop(sf::IntRect({0, 0}, {32, 33})),
                    std::out_of_range);
  BOOST_CHECK_THROW(view.crop(sf::IntRect({-1, 0}, {20, 20})),
                    std::out_of_range);
}
```

```
73
74 // No throw test
75 BOOST_AUTO_TEST_CASE(TestNoThrow) {
76   Image img("images/base.png");
77   SpriteSheet sheet(img, {31, 32}, 1);
78
79   BOOST_CHECK_NO_THROW(sheet[0]);
80   BOOST_CHECK_NO_THROW(sheet[1]);
81
82   TextureView view = sheet[0];
83   BOOST_CHECK_NO_THROW(view.crop(sf::IntRect({5, 5}, {20, 20})));
84 }
85
86 // No margin test
87 BOOST_AUTO_TEST_CASE(TestNoMargin) {
88   Image img("images/base.png");
89
90   SpriteSheet NoMargin(img, {31, 32}, 0);
91   SpriteSheet Margin(img, {31, 32}, 1);
92   sf::IntRect noMarginRect = NoMargin[1].toSprite().getTextureRect();
93   sf::IntRect marginRect = Margin[1].toSprite().getTextureRect();
94   BOOST_CHECK(noMarginRect != marginRect);
95 }
96
97 sf::Image renderToImage(const sb::SpriteBuilder& builder) {
98   sf::RenderTexture renderTexture({64, 64});
99   renderTexture.clear(sf::Color::Transparent);
100  renderTexture.draw(builder);
101  renderTexture.display();
102  return renderTexture.getTexture().copyToImage();
103 }
104
105 bool imagesEqual(const sf::Image& img1, const sf::Image& img2) {
106   if (img1.getSize() != img2.getSize()) return false;
107   for (unsigned int y = 0; y < img1.getSize().y; ++y) {
108     for (unsigned int x = 0; x < img1.getSize().x; ++x) {
109       if (img1.getPixel(sf::Vector2u(x, y)) !=
110           img2.getPixel(sf::Vector2u(x, y))) {
111         return false;
112       }
113     }
114   }
115   return true;
116 }
117
118 static std::set<std::vector<size_t>> collectDistinctSelections(
119     const std::vector<size_t>& layerLengths, int numSeeds) {
120   std::set<std::vector<size_t>> seen;
121   for (int s = 0; s < numSeeds; ++s) {
122     std::mt19937 rng(static_cast<uint32_t>(s));
123     std::vector<size_t> selection;
124     for (size_t len : layerLengths) {
125       std::uniform_int_distribution<size_t> dist(0, len - 1);
126       selection.push_back(dist(rng));
127     }
128     seen.insert(selection);
129   }
130   return seen;
131 }
132
133 // Fixed first seed test
134 BOOST_AUTO_TEST_CASE(TestFixedFirstSeed) {
135   sb::SpriteBuilder builder1("rogue.txt", 12345);
136   sb::SpriteBuilder builder2("rogue.txt", 12345);
137   BOOST_CHECK(imagesEqual(renderToImage(builder1), renderToImage(builder2)));
138
139   sb::SpriteBuilder diff1("rogue.txt", 11111);
140   sb::SpriteBuilder diff2("rogue.txt", 22222);
```

```
  BOOST_CHECK(!imagesEqual(renderToImage(diff1), renderToImage(diff2)));

  // Catches hardcoded default seed: seeded builder must differ from default
  sb::SpriteBuilder seeded("rogue.txt", 12345);
  sb::SpriteBuilder defaultBuilt("rogue.txt");
  BOOST_CHECK(!imagesEqual(renderToImage(seeded), renderToImage(defaultBuilt)));
}

// Inconsistent images test
BOOST_AUTO_TEST_CASE(TestInconsistentImages) {
  sb::SpriteBuilder builder1("rogue.txt", 123);
  sb::SpriteBuilder builder2("rogue.txt", 234);
  BOOST_CHECK(!imagesEqual(renderToImage(builder1), renderToImage(builder2)));
}

// All tiles reachable in single layer
BOOST_AUTO_TEST_CASE(TestGenerateAllSingleLayer) {
  sf::Image img("images/base.png");
  sb::SpriteSheet sheet(img, {16, 16}, 1);
  size_t tileCount = sheet.length();
  BOOST_REQUIRE_GT(tileCount, 0);

  {
    std::ofstream cfg("single_layer.txt");
    cfg << "16 16 1\nimages/base.png\n";
  }

  auto seen = collectDistinctSelections({tileCount}, tileCount * 100);
  BOOST_CHECK_EQUAL(seen.size(), tileCount);
  std::remove("single_layer.txt");
}

// same count layers
BOOST_AUTO_TEST_CASE(TestGenerateAllSameCountLayers) {
  sf::Image img1("images/base.png");
  sf::Image img2("images/pants.png");
  sb::SpriteSheet sheet1(img1, {16, 16}, 1);
  sb::SpriteSheet sheet2(img2, {16, 16}, 1);

  size_t count1 = sheet1.length();
  size_t count2 = sheet2.length();
  BOOST_REQUIRE_GT(count1, 0);
  BOOST_REQUIRE_GT(count2, 0);

  {
    std::ofstream cfg("same_count.txt");
    cfg << "16 16 1\nimages/base.png\nimages/pants.png\n";
  }

  size_t expectedCombos = count1 * count2;
  auto seen = collectDistinctSelections({count1, count2}, expectedCombos * 100);
  BOOST_CHECK_GE(seen.size(), expectedCombos);
  std::remove("same_count.txt");
}

// different count layers
BOOST_AUTO_TEST_CASE(TestGenerateAllDifferentCountLayers) {
  sf::Image img1("images/base.png");
  sf::Image img2("images/pants.png");
  sb::SpriteSheet sheet1(img1, {16, 16}, 1);
  sb::SpriteSheet sheet2(img2, {16, 16}, 1);

  size_t count1 = sheet1.length();
  size_t count2 = sheet2.length();
  BOOST_REQUIRE_GT(count1, 0);
  BOOST_REQUIRE_GT(count2, 0);
  BOOST_CHECK(count1 != count2);

```

```
  {
    std::ofstream cfg("diff_count.txt");
    cfg << "16 16 1\nimages/base.png\nimages/pants.png\n";
  }

  size_t expectedCombos = count1 * count2;
  auto seen = collectDistinctSelections({count1, count2}, expectedCombos * 100);
  BOOST_CHECK_GE(seen.size(), expectedCombos);
  std::remove("diff_count.txt");
}

// Inconsistent subsequent images test
BOOST_AUTO_TEST_CASE(TestInconsistentSubsequentImages) {
  sb::SpriteBuilder builder1("rogue.txt", 99999);
  sb::SpriteBuilder builder2("rogue.txt", 99999);

  BOOST_CHECK(imagesEqual(renderToImage(builder1), renderToImage(builder2)));

  builder1.randomize();
  builder2.randomize();
  BOOST_CHECK(imagesEqual(renderToImage(builder1), renderToImage(builder2)));

  builder1.randomize();
  builder2.randomize();
  BOOST_CHECK(imagesEqual(renderToImage(builder1), renderToImage(builder2)));

  sb::SpriteBuilder builder3("rogue.txt", 99999);
  sf::Image originalImage = renderToImage(builder3);
  builder3.randomize();
  sf::Image randomizedImage = renderToImage(builder3);
  BOOST_CHECK(!imagesEqual(originalImage, randomizedImage));
}
```

3 PS2: Recursive Tree

3.1 Assignment Overview

Build a Pythagorean tree fractal where each branch sprouts two smaller branches that together with the parent form a right triangle. The program is run as ./PTree L N where L is the initial side length and N is the recursion depth. The window automatically scales so the full tree fits on screen regardless of depth. As extra credit, the branches shift color from green at the trunk to pink at the tips.

3.2 Key Concepts and Design Choices

The algorithm is recursive: given the four corners of a square, compute the apex of the right triangle on top, then recursively draw two child squares on the two shorter sides. Each recursive call applies an sf::Transform that rotates and scales relative to the parent so child branches are correctly positioned without manually tracking absolute world coordinates. Window scaling computes the bounding box of the full tree at depth N and fits it to the window with a uniform scale factor.

3.3 What I Learned

I learned how sf::Transform composes, passing transforms down the recursive stack is equivalent to multiplying matrices, so each child automatically inherits the cumulative rotation and scale of all its ancestors. Using the graphics library's transform stack is far more efficient than computing absolute positions by hand.

3.4 Evidence

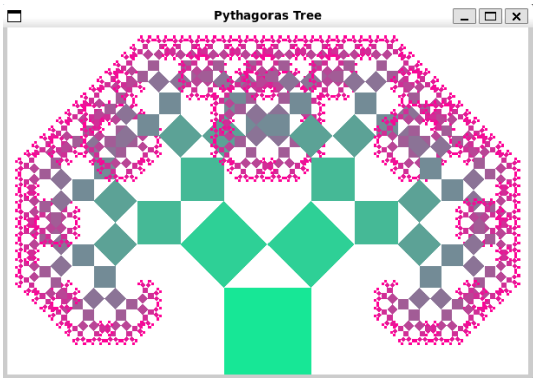


Figure 3: PS2: Pythagorean tree fractal with depth-based color gradient

3.5 Incomplete Features

The trickiest bug was accidentally using int arithmetic for the triangle apex coordinates instead of float. Using Int misaligned branches at larger depths and eventually crashed at deep recursion due to accumulated rounding. Switching every coordinate to float fixed both issues. Very high recursion depths still cause a stack overflow, which is a limitation of the recursive approach.

3.6 Source Code

Makefile

```
CC = g++
CFLAGS = -std=c++17 -Wall -Werror -pedantic
LIB = -lsfml-graphics -lsfml-window -lsfml-system
```

```
4
5 all: PTree
6
7 PTree: main.o PTree.o
8   $(CC) $(CFLAGS) -o PTree main.o PTree.o $(LIB)
9
10 main.o: main.cpp PTree.hpp
11   $(CC) $(CFLAGS) -c main.cpp
12
13 PTree.o: PTree.cpp PTree.hpp
14   $(CC) $(CFLAGS) -c PTree.cpp
15
16 lint:
17   cpplint *.cpp *.hpp
18
19 clean:
20   rm -f *.o PTree
```

main.cpp

```
1 // Copyright 2026 Benny Chen
2 #include <iostream>
3 #include <optional>
4 #include "PTree.hpp"
5 #include <SFML/Graphics.hpp>
6
7 int main(int argc, char* argv[]) {
8     if (argc != 3) {
9         std::cerr << "Usage: " << argv[0] << " L N" << std::endl;
10        return -1;
11    }
12
13    double L = std::stod(argv[1]);
14    int N = std::stoi(argv[2]);
15
16    unsigned int width = static_cast<unsigned int>(6 * L);
17    unsigned int height = static_cast<unsigned int>(4 * L);
18
19    sf::RenderWindow window(sf::VideoMode({width, height}), "Pythagoras Tree");
20
21    PTree tree(L, N);
22
23    while (window.isOpen()) {
24        while (const std::optional event = window.pollEvent()) {
25            if (event->is<sf::Event::Closed>())
26                window.close();
27
28            if (const auto* key = event->getIf<sf::Event::KeyPressed>())
29                if (key->code == sf::Keyboard::Key::Escape)
30                    window.close();
31        }
32
33        window.clear(sf::Color::White);
34        window.draw(tree);
35        window.display();
36    }
37
38    return 0;
39 }
```

PTree.hpp

```
1 // Copyright 2026 Benny Chen
2 #ifndef PTREE_HPP
```

```
#define PTREE_HPP

#include <cstdint>
#include <cmath>
#include <vector>
#include <SFML/Graphics.hpp>

class PTree : public sf::Drawable {
 public:
    PTree(double L, int N);
    void pTree(sf::RenderTarget& target,
        sf::RenderStates states,
        double size, int depth,
        sf::Transform transform
) const;

 private:
    void draw(sf::RenderTarget& target, sf::RenderStates states) const override;
    double baseSize;
    int maxDepth;
};

#endif
```

PTree.cpp

```
// Copyright 2026 Benny Chen
#include "PTree.hpp"

PTree::PTree(double L, int N) : baseSize(L), maxDepth(N) {}

void PTree::draw(sf::RenderTarget& target, sf::RenderStates states) const {
    sf::Transform iTransform;
    iTransform.translate({static_cast<float>(3.0 * baseSize - baseSize / 2.0),
                          static_cast<float>(4.0 * baseSize - baseSize)});

    pTree(target, states, baseSize, maxDepth, iTransform);
}

void PTree::pTree(sf::RenderTarget& target,
    sf::RenderStates states,
    double size, int depth,
    sf::Transform transform
) const {
    if (depth < 0) return;

    sf::RectangleShape square({static_cast<float>(size), static_cast<float>(size)});

    std::uint8_t r = static_cast<std::uint8_t>(
        255 * (1.0 - static_cast<double>(depth) / (maxDepth + 1)));
    std::uint8_t g = static_cast<std::uint8_t>(
        255 * (static_cast<double>(depth) / (maxDepth + 1)));
    std::uint8_t b = 150;
    square.setFillColor(sf::Color(r, g, b));

    target.draw(square, transform);

    if (depth > 0) {
        double nextSize = size * std::sqrt(2.0) / 2.0;

        // Left child
        sf::Transform leftTransform = transform;
        leftTransform.rotate(sf::degrees(-45.0f), {0.0f, 0.0f});
        leftTransform.translate({0.0f, static_cast<float>(-nextSize)});
        pTree(target, states, nextSize, depth - 1, leftTransform);

```

```
41          // Right child
42          sf::Transform rightTransform = transform;
43          rightTransform.translate({static_cast<float>(size), 0.0f});
44          rightTransform.rotate(sf::degrees(45.0f), {0.0f, 0.0f});
45          rightTransform.translate({static_cast<float>(-nextSize), static_cast<float>(-nextSize)});
46          pTree(target, states, nextSize, depth - 1, rightTransform);
47      }
48  }
```

4 PS3: Sokoban

4.1 Assignment Overview

PS3 implemented the classic Sokoban puzzle game in two parts. Part A handled level loading and displaying puzzle levels. The program reads a level file containing grid dimensions and symbols which represents game elements. Representations are player(@), walls(#), boxes(A), storage(a), and empty spaces(.). It displays the static, without movement functionality, game board with appropiate sprites. Part b adds game mechanics to the static display from part a. Implements player movement using Keyboard input (WASD or arrow keys). Player is able to push boxes one at a time, cannot push though walls, and must detect when a box is on storage location. A win condition and number of moves is executed when all boxes are placed on storage.

4.2 Key Concepts and Design Choices

The drawing system uses nested for loops over the grid, drawing each tile in passes. First, the floor/storage layer, then walls, boxes, and the player on top. Inside the loop, if/else branches select the right texture rect based on the current tile type. Game state is stored in std::vector¡TileType¿. A separate sf::Vector2u tracks the player's position, and a std::vector stores all storage goal locations. Movement logic is in movePlayer(), which checks boundaries and walls before allowing a move. If the player attempts to push a box, the tile two steps ahead must be free. The win condition is checked by counting BoxOnStorage tiles against the total number of storage locations.

4.3 What I Learned

The main challenge was keeping track of overlapping tile states: a tile can be a storage location and also have a box on it at the same time. I used an enum with separate BOX_ON_STORAGE and PLAYER_ON_STORAGE values instead of two layers, which kept rendering simpler but meant more cases to handle during movement.

4.4 Evidence

Figure 4: PS3: Sokoban level loaded and playable

4.5 Incomplete Features

The lambda function for the drawing system wasn't fully implemented. I ran out of time and didn't have a solid enough understanding of lambdas yet to get it working correctly.

4.6 Source Code

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Werror -pedantic
LDFLAGS = -lsfml-graphics -lsfml-window -lsfml-system
TEST_LDFLAGS = -lboost_unit_test_framework

TARGET_MAIN = Sokoban
TARGET_LIB = Sokoban.a
TARGET_TEST = test

all: $(TARGET_MAIN) $(TARGET_LIB) $(TARGET_TEST)

$(TARGET_MAIN): main.o Sokoban.o
  $(CXX) $(CXXFLAGS) main.o Sokoban.o -o $(TARGET_MAIN) $(LDFLAGS)

$(TARGET_LIB): Sokoban.o
  ar rcs $(TARGET_LIB) Sokoban.o

$(TARGET_TEST): test.o Sokoban.o
  $(CXX) $(CXXFLAGS) test.o Sokoban.o -o $(TARGET_TEST) $(TEST_LDFLAGS) $(LDFLAGS)

main.o: main.cpp Sokoban.hpp
  $(CXX) $(CXXFLAGS) -c main.cpp -o main.o

Sokoban.o: Sokoban.cpp Sokoban.hpp
  $(CXX) $(CXXFLAGS) -c Sokoban.cpp -o Sokoban.o

test.o: test.cpp Sokoban.hpp
  $(CXX) $(CXXFLAGS) -c test.cpp -o test.o

lint:
  cpplint --filter=-legal/copyright main.cpp Sokoban.cpp Sokoban.hpp test.cpp

clean:
  rm -f *.o $(TARGET_MAIN) $(TARGET_LIB) $(TARGET_TEST)

.PHONY: all lint clean
```

main.cpp

```
// Copyright 2026 Benny Chen

#include <iostream>

#include <SFML/Graphics.hpp>

#include "Sokoban.hpp"

int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " <level_file>" << std::endl;
        return 1;
    }

    SB::Sokoban game(argv[1]);

    if (game.width() == 0 || game.height() == 0) {
        std::cerr << "Error: Could not load level from " << argv[1] << std::endl;
        return 1;
    }

    sf::RenderWindow window(sf::VideoMode(game.windowSize()), "Sokoban");
```

```
    window.setFramerateLimit(60);

    while (window.isOpen()) {
        while (const std::optional<sf::Event> event = window.pollEvent()) {
            if (event->is<sf::Event::Closed>()) {
                window.close();
            }

            if (const auto* keyPressed = event->getIf<sf::Event::KeyPressed>()) {
                switch (keyPressed->code) {
                    case sf::Keyboard::Key::W:
                    case sf::Keyboard::Key::Up:
                        game.movePlayer(SB::Direction::Up);
                        break;
                    case sf::Keyboard::Key::S:
                    case sf::Keyboard::Key::Down:
                        game.movePlayer(SB::Direction::Down);
                        break;
                    case sf::Keyboard::Key::A:
                    case sf::Keyboard::Key::Left:
                        game.movePlayer(SB::Direction::Left);
                        break;
                    case sf::Keyboard::Key::D:
                    case sf::Keyboard::Key::Right:
                        game.movePlayer(SB::Direction::Right);
                        break;
                    case sf::Keyboard::Key::R:
                        game.reset();
                        break;
                    case sf::Keyboard::Key::Z:
                        game.undo();
                        break;
                    case sf::Keyboard::Key::Y:
                        game.redo();
                        break;
                    default:
                        break;
                }
            }
        }

        window.clear();
        window.draw(game);
        window.display();
    }

    return 0;
}
```

SpriteSheet.hpp

```
// Copyright 2026 Benny
#pragma once

#include <memory>
#include <string>

#include <SFML/Graphics.hpp>

namespace sb {

class TextureView {
 public:
  TextureView(std::shared_ptr<sf::Texture> texture, sf::IntRect region);

  TextureView crop(sf::IntRect area) const;
```

```
16
17   sf::Vector2u getSize() const;
18   sf::Sprite toSprite() const;
19   sf::Image toImage() const;
20
21  private:
22   std::shared_ptr<sf::Texture> texture_;
23   sf::IntRect region_;
24 };
25
26 class SpriteSheet {
27  public:
28   SpriteSheet(sf::Image img, sf::Vector2u tileSize, int margin = 0);
29   SpriteSheet(const std::string &imgFilename, sf::Vector2u tileSize,
30               int margin = 0);
31
32   sf::Vector2u getTileSize() const;
33   sf::Vector2u size() const;
34
35   size_t length() const;
36   size_t width() const;
37   size_t height() const;
38
39   TextureView operator[](sf::Vector2u pt) const;
40   TextureView operator[](size_t i) const;
41
42  private:
43   std::shared_ptr<sf::Texture> texture_;
44   sf::Vector2u tileSize_;
45   int margin_;
46   size_t gridWidth_;
47   size_t gridHeight_;
48 };
49
50 }
51 // namespace sb
```

SpriteSheet.cpp

```
1  // Copyright 2026 Benny
2
3  #include "SpriteSheet.hpp"
4  #include <stdexcept>
5
6  namespace sb {
7
8  // texture view
9  TextureView::TextureView(std::shared_ptr<sf::Texture> texture,
10                          sf::IntRect region)
11     : texture_(texture), region_(region) {
12   if (!texture_) {
13     throw std::invalid_argument("Texture cannot be null");
14   }
15
16   // check region in bound
17   sf::Vector2u texSize = texture_->getSize();
18   if (region_.position.x < 0 || region_.position.y < 0 ||
19       region_.size.x < 0 || region_.size.y < 0 ||
20       static_cast<unsigned>(region_.position.x + region_.size.x) > texSize.x ||
21       static_cast<unsigned>(region_.position.y + region_.size.y) > texSize.y) {
22     throw std::out_of_range("Region extends outside texture bounds");
23   }
24 }
25
26 // crop area
27 TextureView TextureView::crop(sf::IntRect area) const {
```

```
  if (area.position.x < 0 || area.position.y < 0 || area.size.x < 0 ||
      area.size.y < 0 || area.position.x + area.size.x > region_.size.x ||
      area.position.y + area.size.y > region_.size.y) {
    throw std::out_of_range("Crop area extends outside view bounds");
  }

  sf::IntRect absoluteRegion(sf::Vector2i(region_.position.x + area.position.x,
                                          region_.position.y + area.position.y),
                             area.size);

  return TextureView(texture_, absoluteRegion);
}

sf::Vector2u TextureView::getSize() const {
  return sf::Vector2u(static_cast<unsigned>(region_.size.x),
                      static_cast<unsigned>(region_.size.y));
}

sf::Sprite TextureView::toSprite() const {
  sf::Sprite sprite(*texture_);
  sprite.setTextureRect(region_);
  return sprite;
}

// create image of texture
sf::Image TextureView::toImage() const {
  sf::Image fullImage = texture_->copyToImage();
  sf::Image result;
  result.resize(getSize());

  for (unsigned int y = 0; y < getSize().y; ++y) {
    for (unsigned int x = 0; x < getSize().x; ++x) {
      sf::Color pixel = fullImage.getPixel(
          sf::Vector2u(static_cast<unsigned>(region_.position.x) + x,
                       static_cast<unsigned>(region_.position.y) + y));
      result.setPixel(sf::Vector2u(x, y), pixel);
    }
  }

  return result;
}

// sprite sheet
SpriteSheet::SpriteSheet(sf::Image img, sf::Vector2u tileSize, int margin)
    : tileSize_(tileSize), margin_(margin) {
  if (tileSize.x == 0 || tileSize.y == 0) {
    throw std::invalid_argument("Tile size must be greater than 0");
  }

  if (margin < 0) {
    throw std::invalid_argument("Margin cannot be negative");
  }

  texture_ = std::make_shared<sf::Texture>();
  if (!texture_->loadFromImage(img)) {
    throw std::runtime_error("Failed to load texture from image");
  }

  sf::Vector2u imageSize = img.getSize();

  if (imageSize.x < static_cast<unsigned>(margin) + tileSize.x) {
    gridWidth_ = 0;
  } else {
    gridWidth_ = (imageSize.x + margin) / (tileSize.x + margin);
  }

  if (imageSize.y < static_cast<unsigned>(margin) + tileSize.y) {
    gridHeight_ = 0;
```

```
  } else {
    gridHeight_ = (imageSize.y + margin) / (tileSize.y + margin);
  }

  if (gridWidth_ == 0 || gridHeight_ == 0) {
    throw std::invalid_argument(
        "Image too small for given tile size and margin");
  }
}

SpriteSheet::SpriteSheet(const std::string &imgFilename, sf::Vector2u tileSize,
                         int margin)
    : tileSize_(tileSize), margin_(margin) {
  sf::Image img;
  if (!img.loadFromFile(imgFilename)) {
    throw std::runtime_error("Failed to load image from file: " + imgFilename);
  }

  if (tileSize.x == 0 || tileSize.y == 0) {
    throw std::invalid_argument("Tile size must be greater than 0");
  }

  if (margin < 0) {
    throw std::invalid_argument("Margin cannot be negative");
  }

  texture_ = std::make_shared<sf::Texture>();
  if (!texture_->loadFromImage(img)) {
    throw std::runtime_error("Failed to load texture from image");
  }

  sf::Vector2u imageSize = img.getSize();

  if (imageSize.x < static_cast<unsigned>(margin) + tileSize.x) {
    gridWidth_ = 0;
  } else {
    gridWidth_ = (imageSize.x - margin) / (tileSize.x + margin);
  }

  if (imageSize.y < static_cast<unsigned>(margin) + tileSize.y) {
    gridHeight_ = 0;
  } else {
    gridHeight_ = (imageSize.y - margin) / (tileSize.y + margin);
  }

  if (gridWidth_ == 0 || gridHeight_ == 0) {
    throw std::invalid_argument(
        "Image too small for given tile size and margin");
  }
}

sf::Vector2u SpriteSheet::getTileSize() const { return tileSize_; }

size_t SpriteSheet::width() const { return gridWidth_; }

size_t SpriteSheet::height() const { return gridHeight_; }

sf::Vector2u SpriteSheet::size() const {
  return sf::Vector2u(static_cast<unsigned>(gridWidth_),
                      static_cast<unsigned>(gridHeight_));
}

size_t SpriteSheet::length() const { return gridWidth_ * gridHeight_; }

TextureView SpriteSheet::operator[](sf::Vector2u pt) const {
  if (pt.x >= gridWidth_ || pt.y >= gridHeight_) {
    throw std::out_of_range("Grid position out of bounds");
  }
```

```

  int pixelX = margin_ + static_cast<int>(pt.x) *
                         (static_cast<int>(tileSize_.x) + margin_);
  int pixelY = margin_ + static_cast<int>(pt.y) *
                         (static_cast<int>(tileSize_.y) + margin_);
  int tileWidth = static_cast<int>(tileSize_.x);
  int tileHeight = static_cast<int>(tileSize_.y);

  return TextureView(texture_,
                  sf::IntRect(sf::Vector2i(pixelX, pixelY),
                              sf::Vector2i(tileWidth, tileHeight)));
}

TextureView SpriteSheet::operator[](size_t i) const {
  if (i >= length()) {
    throw std::out_of_range("Index out of bounds");
  }

  unsigned int x = i % gridWidth_;
  unsigned int y = i / gridWidth_;

  return (*this)[sf::Vector2u(x, y)];
}

} // namespace sb
```

Sokoban.hpp

```
// Copyright 2026 Benny Chen

#pragma once

#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <optional>
#include <stack>

#include <SFML/Graphics.hpp>

namespace SB {
enum class Direction {
    Up, Down, Left, Right
};

enum class TileType {
    Empty = ' ',
    Wall = '#',
    Box = 'A',
    Storage = 'a',
    Player = '@',
    BoxOnStorage = '1',
    PlayerOnStorage = '+'
};

class Sokoban : public sf::Drawable {
 public:
    static const int TILE_SIZE = 64;

    Sokoban();
    explicit Sokoban(const std::string&);

    unsigned int height() const;
    unsigned int width() const;

```

```
    sf::Vector2u size() const;
    sf::Vector2u windowSize() const;
    sf::Vector2u playerLoc() const;

    bool isWon() const;

    void movePlayer(Direction dir);
    void reset();

    void undo();
    void redo();

protected:
    void draw(sf::RenderTarget& target, sf::RenderStates states) const override;

private:
    struct GameState {
        sf::Vector2u playerLoc;
        std::vector<TileType> grid;
    };

    unsigned int height_;
    unsigned int width_;
    sf::Vector2u playerLoc_;
    std::vector<TileType> grid_;
    std::vector<sf::Vector2u> storageLocations_;

    GameState initialState_;

    std::stack<GameState> undoStack_;
    std::stack<GameState> redoStack_;

    sf::Texture tilesheet_;
    std::map<TileType, sf::IntRect> tileRects_;
    mutable std::optional<sf::Sprite> sprite_;

    sf::Font font_;
    mutable sf::Text winText_;

    void loadTextures();
    void loadFont();
    size_t getIndex(unsigned int x, unsigned int y) const { return y * width_ + x; }
    GameState captureState() const;
    void restoreState(const GameState& state);

    friend std::ostream& operator<<(std::ostream& out, const Sokoban& s);
    friend std::istream& operator>>(std::istream& in, Sokoban& s);
};

} // namespace SB
```

Sokoban.cpp

```
// Copyright 2026 Benny Chen

#include "Sokoban.hpp"
#include <fstream>
#include <vector>
#include <string>
#include <iostream>
#include <algorithm>

namespace SB {

Sokoban::Sokoban() : height_(0), width_(0), winText_(font_) {
    loadTextures();
```

```
14      loadFont();
15  }
16
17  Sokoban::Sokoban(const std::string& filename) : height_(0), width_(0), winText_(font_) {
18      loadTextures();
19      loadFont();
20      std::ifstream file(filename);
21      if (!file.is_open()) {
22          std::cerr << "Error: Could not open level file "
23          << filename << std::endl;
24          return;
25      }
26      file >> *this;
27  }
28
29  void Sokoban::loadFont() {
30      font_.openFromFile("arial.ttf") ||
31      font_.openFromFile("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf") ||
32      font_.openFromFile("/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf");
33
34      winText_.setFont(font_);
35      winText_.setString("You Win!");
36      winText_.setCharacterSize(72);
37      winText_.setFillColor(sf::Color::Yellow);
38      winText_.setOutlineThickness(4);
39      winText_.setStyle(sf::Text::Bold);
40  }
41
42  void Sokoban::loadTextures() {
43      if (!tilesheet_.loadFromFile("sokoban_tilesheet.png")) {
44          std::cerr << "Error: Could not load sokoban_tilesheet.png" << std::endl;
45          return;
46      }
47      sprite_.emplace(tilesheet_);
48
49      // floor
50      tileRects_[TileType::Empty] = sf::IntRect(
51          {12 * TILE_SIZE, 6 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
52
53      tileRects_[TileType::Wall] = sf::IntRect(
54          {1 * TILE_SIZE, 0 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
55
56      tileRects_[TileType::Box] = sf::IntRect(
57          {7 * TILE_SIZE, 0 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
58
59      tileRects_[TileType::Storage] = sf::IntRect(
60          {0 * TILE_SIZE, 3 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
61
62      tileRects_[TileType::Player] = sf::IntRect(
63          {0 * TILE_SIZE, 5 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
64
65      tileRects_[TileType::PlayerOnStorage] = sf::IntRect(
66          {0 * TILE_SIZE, 5 * TILE_SIZE}, {TILE_SIZE, TILE_SIZE});
67  }
68
69  unsigned int Sokoban::height() const { return height_; }
70  unsigned int Sokoban::width() const { return width_; }
71
72  sf::Vector2u Sokoban::size() const {
73      return {width_, height_};
74  }
75
76  sf::Vector2u Sokoban::windowSize() const {
77      return {width_ * TILE_SIZE, height_ * TILE_SIZE};
78  }
79
80  sf::Vector2u Sokoban::playerLoc() const { return playerLoc_; }
81
```

```
82 bool Sokoban::isWon() const {
83     int boxesOnStorage = std::count(grid_.begin(), grid_.end(), TileType::BoxOnStorage);
84     int totalBoxes = boxesOnStorage +
85         std::count(grid_.begin(), grid_.end(), TileType::Box);
86     int totalStorage = static_cast<int>(storageLocations_.size());
87     return boxesOnStorage >= std::min(totalBoxes, totalStorage);
88 }
89
90 Sokoban::GameState Sokoban::captureState() const {
91     return {playerLoc_, grid_};
92 }
93
94 void Sokoban::restoreState(const GameState& state) {
95     playerLoc_ = state.playerLoc;
96     grid_ = state.grid;
97 }
98
99 void Sokoban::movePlayer(Direction dir) {
100     if (isWon()) return;
101
102     int dx = 0, dy = 0;
103     switch (dir) {
104         case Direction::Up: dy = -1; break;
105         case Direction::Down: dy = 1; break;
106         case Direction::Left: dx = -1; break;
107         case Direction::Right: dx = 1; break;
108     }
109
110     int px = static_cast<int>(playerLoc_.x);
111     int py = static_cast<int>(playerLoc_.y);
112
113     int nx = px + dx;
114     int ny = py + dy;
115
116     if (nx < 0 || ny < 0 ||
117         static_cast<unsigned>(nx) >= width_ ||
118         static_cast<unsigned>(ny) >= height_) {
119         return;
120     }
121
122     TileType dest = grid_[getIndex(nx, ny)];
123
124     if (dest == TileType::Wall) return;
125     if (dest == TileType::Box || dest == TileType::BoxOnStorage) {
126         int bx = nx + dx;
127         int by = ny + dy;
128
129         if (bx < 0 || by < 0 ||
130             static_cast<unsigned>(bx) >= width_ ||
131             static_cast<unsigned>(by) >= height_) {
132             return;
133         }
134         TileType beyondBox = grid_[getIndex(bx, by)];
135
136         if (beyondBox == TileType::Wall ||
137             beyondBox == TileType::Box ||
138             beyondBox == TileType::BoxOnStorage) {
139             return;
140         }
141
142         // Save state before move
143         undoStack_.push(captureState());
144         while (!redoStack_.empty()) redoStack_.pop();
145
146         bool boxWasOnStorage = (dest == TileType::BoxOnStorage);
147         bool beyondIsStorage = (beyondBox == TileType::Storage);
148
149         grid_[getIndex(bx, by)] = beyondIsStorage ? TileType::BoxOnStorage : TileType::Box;
```

```
150
151         TileType playerTile = grid_[getIndex(px, py)];
152         grid_[getIndex(px, py)] = (playerTile == TileType::PlayerOnStorage)
153             ? TileType::Storage : TileType::Empty;
154
155         grid_[getIndex(nx, ny)] = boxWasOnStorage
156             ? TileType::PlayerOnStorage : TileType::Player;
157
158         playerLoc_ = {static_cast<unsigned>(nx), static_cast<unsigned>(ny)};
159
160     } else {
161         undoStack_.push(captureState());
162         while (!redoStack_.empty()) redoStack_.pop();
163
164         TileType playerTile = grid_[getIndex(px, py)];
165         grid_[getIndex(px, py)] = (playerTile == TileType::PlayerOnStorage)
166             ? TileType::Storage : TileType::Empty;
167
168         bool destIsStorage = (dest == TileType::Storage);
169         grid_[getIndex(nx, ny)] = destIsStorage
170             ? TileType::PlayerOnStorage : TileType::Player;
171
172         playerLoc_ = {static_cast<unsigned>(nx), static_cast<unsigned>(ny)};
173     }
174 }
175
176 void Sokoban::reset() {
177     restoreState(initialState_);
178     while (!undoStack_.empty()) undoStack_.pop();
179     while (!redoStack_.empty()) redoStack_.pop();
180 }
181
182 void Sokoban::undo() {
183     if (undoStack_.empty()) return;
184     redoStack_.push(captureState());
185     restoreState(undoStack_.top());
186     undoStack_.pop();
187 }
188
189 void Sokoban::redo() {
190     if (redoStack_.empty()) return;
191     undoStack_.push(captureState());
192     restoreState(redoStack_.top());
193     redoStack_.pop();
194 }
195
196 void Sokoban::draw(sf::RenderTarget& target, sf::RenderStates states) const {
197     if (!sprite_) return;
198
199     for (unsigned int y = 0; y < height_; ++y) {
200         for (unsigned int x = 0; x < width_; ++x) {
201             sf::Vector2f position(
202                 static_cast<float>(x * TILE_SIZE),
203                 static_cast<float>(y * TILE_SIZE));
204
205             TileType currentTile = grid_[getIndex(x, y)];
206
207             sprite_->setTextureRect(tileRects_.at(TileType::Empty));
208             sprite_->setPosition(position);
209             target.draw(*sprite_, states);
210
211             bool onStorage = (currentTile == TileType::Storage ||
212                               currentTile == TileType::BoxOnStorage ||
213                               currentTile == TileType::PlayerOnStorage);
214             if (onStorage) {
215                 sprite_->setTextureRect(tileRects_.at(TileType::Storage));
216                 sprite_->setPosition(position);
217                 target.draw(*sprite_, states);
```

```
218             }
219
220             if (currentTile == TileType::Wall) {
221                 sprite_->setTextureRect(tileRects_.at(TileType::Wall));
222                 sprite_->setPosition(position);
223                 target.draw(*sprite_, states);
224             } else if (currentTile == TileType::Box) {
225                 sprite_->setTextureRect(tileRects_.at(TileType::Box));
226                 sprite_->setPosition(position);
227                 target.draw(*sprite_, states);
228             } else if (currentTile == TileType::BoxOnStorage) {
229                 sprite_->setTextureRect(tileRects_.at(TileType::Box));
230                 sprite_->setPosition(position);
231                 target.draw(*sprite_, states);
232             }
233         }
234     }
235
236     // player on top
237     TileType tileAtPlayerLoc = grid_[getIndex(playerLoc_.x, playerLoc_.y)];
238     sprite_->setTextureRect(
239         tileRects_.at(tileAtPlayerLoc == TileType::PlayerOnStorage
240             ? TileType::PlayerOnStorage
241             : TileType::Player));
242     sprite_->setPosition(
243         {static_cast<float>(playerLoc_.x * TILE_SIZE),
244          static_cast<float>(playerLoc_.y * TILE_SIZE)});
245     target.draw(*sprite_, states);
246
247     // win
248     if (isWon()) {
249         sf::RectangleShape overlay(
250             {static_cast<float>(width_ * TILE_SIZE),
251              static_cast<float>(height_ * TILE_SIZE)});
252         overlay.setFillColor(sf::Color(0, 0, 0, 150));
253         target.draw(overlay, states);
254
255         sf::FloatRect textBounds = winText_.getLocalBounds();
256         winText_.setOrigin({textBounds.position.x + textBounds.size.x / 2.f,
257                             textBounds.position.y + textBounds.size.y / 2.f});
258         winText_.setPosition(
259             {static_cast<float>(width_ * TILE_SIZE) / 2.f,
260              static_cast<float>(height_ * TILE_SIZE) / 2.f});
261         target.draw(winText_, states);
262     }
263 }
264
265 std::istream& operator>>(std::istream& in, Sokoban& s) {
266     if (!(in >> s.height_ >> s.width_)) return in;
267     in.ignore();
268
269     s.grid_.clear();
270     s.grid_.resize(s.width_ * s.height_);
271     s.storageLocations_.clear();
272
273     std::string line;
274     for (unsigned int y = 0; y < s.height_; ++y) {
275         if (!std::getline(in, line)) break;
276         for (unsigned int x = 0; x < s.width_ && x < line.length(); ++x) {
277             char tileChar = line[x];
278             TileType type;
279             switch (tileChar) {
280                 case '@':
281                     type = TileType::Player;
282                     s.playerLoc_ = {x, y};
283                     break;
284                 case '.': type = TileType::Empty; break;
285                 case '#': type = TileType::Wall; break;
```

```
286                 case 'A': type = TileType::Box; break;
287                 case 'a':
288                     type = TileType::Storage;
289                     s.storageLocations_.push_back({x, y});
290                     break;
291                 case '1':
292                     type = TileType::BoxOnStorage;
293                     s.storageLocations_.push_back({x, y});
294                     break;
295                 case '+':
296                     type = TileType::PlayerOnStorage;
297                     s.playerLoc_ = {x, y};
298                     s.storageLocations_.push_back({x, y});
299                     break;
300                 default: type = TileType::Empty; break;
301             }
302             s.grid_[s.getIndex(x, y)] = type;
303         }
304     }
305
306     // Save initial state for reset
307     s.initialState_ = {s.playerLoc_, s.grid_};
308
309     return in;
310 }
311
312 std::ostream& operator<<(std::ostream& out, const Sokoban& s) {
313     for (unsigned int y = 0; y < s.height_; ++y) {
314         for (unsigned int x = 0; x < s.width_; ++x) {
315             out << static_cast<char>(s.grid_[s.getIndex(x, y)]);
316         }
317         out << std::endl;
318     }
319     return out;
320 }
321
322 } // namespace SB
```

test.cpp

```
1  // Copyright 2026 Benny Chen
2  // Unit tests for Sokoban PS3b
3
4  #define BOOST_TEST_DYN_LINK
5  #define BOOST_TEST_MODULE SokobanTests
6
7  #include <sstream>
8  #include <string>
9  #include <boost/test/unit_test.hpp>
10 #include "Sokoban.hpp"
11
12 static SB::Sokoban makeGame(const std::string& lvlStr) {
13     SB::Sokoban s;
14     std::istringstream iss(lvlStr);
15     iss >> s;
16     return s;
17 }
18
19 // File Parsing
20 BOOST_AUTO_TEST_SUITE(FileParsing)
21
22 BOOST_AUTO_TEST_CASE(LoadDimensionsAndPlayerPos) {
23     std::string lvl =
24         "3 5\n"
25         "#####\n"
26         "#@Aa#\n"
```

```
27          "#####\n";
28      SB::Sokoban s = makeGame(lvl);
29      BOOST_CHECK_EQUAL(s.height(), 3u);
30      BOOST_CHECK_EQUAL(s.width(), 5u);
31      BOOST_CHECK_EQUAL(s.playerLoc().x, 1u);
32      BOOST_CHECK_EQUAL(s.playerLoc().y, 1u);
33  }
34
35  BOOST_AUTO_TEST_SUITE_END()
36
37  // Basic Movement
38  BOOST_AUTO_TEST_SUITE(BasicMovement)
39
40  BOOST_AUTO_TEST_CASE(MoveRight) {
41      std::string lvl =
42          "3 5\n"
43          "#####\n"
44          "#@aA#\n"
45          "#####\n";
46      SB::Sokoban s = makeGame(lvl);
47      s.movePlayer(SB::Direction::Right);
48      BOOST_CHECK_EQUAL(s.playerLoc().x, 2u);
49      BOOST_CHECK_EQUAL(s.playerLoc().y, 1u);
50  }
51
52  BOOST_AUTO_TEST_CASE(MoveLeft) {
53      std::string lvl =
54          "3 5\n"
55          "#####\n"
56          "#aA@#\n"
57          "#####\n";
58      SB::Sokoban s = makeGame(lvl);
59      s.movePlayer(SB::Direction::Left);
60      BOOST_CHECK_EQUAL(s.playerLoc().x, 2u);
61      BOOST_CHECK_EQUAL(s.playerLoc().y, 1u);
62  }
63
64  BOOST_AUTO_TEST_CASE(MoveUp) {
65      std::string lvl =
66          "5 3\n"
67          "###\n"
68          "#a#\n"
69          "#A#\n"
70          "#@#\n"
71          "###\n";
72      SB::Sokoban s = makeGame(lvl);
73      s.movePlayer(SB::Direction::Up);
74      BOOST_CHECK_EQUAL(s.playerLoc().x, 1u);
75      BOOST_CHECK_EQUAL(s.playerLoc().y, 2u);
76  }
77
78  BOOST_AUTO_TEST_CASE(MoveDown) {
79      std::string lvl =
80          "5 3\n"
81          "###\n"
82          "#@#\n"
83          "#A#\n"
84          "#a#\n"
85          "###\n";
86      SB::Sokoban s = makeGame(lvl);
87      s.movePlayer(SB::Direction::Down);
88      BOOST_CHECK_EQUAL(s.playerLoc().x, 1u);
89      BOOST_CHECK_EQUAL(s.playerLoc().y, 2u);
90  }
91
92  BOOST_AUTO_TEST_CASE(PlayerStaysInBoundsRight) {
93      std::string lvl =
94          "3 3\n"
```

```
95          "###\n"
96          "#.@\n"
97          "###\n";
98      SB::Sokoban s = makeGame(lvl);
99      BOOST_REQUIRE_EQUAL(s.playerLoc().x, 2u);
100     s.movePlayer(SB::Direction::Right);
101     BOOST_CHECK_EQUAL(s.playerLoc().x, 2u);
102     BOOST_CHECK(s.playerLoc().x < s.width());
103 }
104
105 BOOST_AUTO_TEST_CASE(PlayerStaysInBoundsUp) {
106     std::string lvl =
107         "3 3\n"
108         ".@.\n"
109         "...\n"
110         "...\n";
111     SB::Sokoban s = makeGame(lvl);
112     BOOST_REQUIRE_EQUAL(s.playerLoc().y, 0u);
113     s.movePlayer(SB::Direction::Up);
114     BOOST_CHECK_EQUAL(s.playerLoc().y, 0u);
115     BOOST_CHECK(s.playerLoc().y < s.height());
116 }
117
118 BOOST_AUTO_TEST_SUITE_END()
119
120 // Box Interactions
121 BOOST_AUTO_TEST_SUITE(BoxTests)
122
123 BOOST_AUTO_TEST_CASE(CannotPushTwoBoxes) {
124     std::string lvl =
125         "3 6\n"
126         "######\n"
127         "#@AAa#\n"
128         "######\n";
129     SB::Sokoban s = makeGame(lvl);
130     sf::Vector2u before = s.playerLoc();
131     s.movePlayer(SB::Direction::Right);
132     BOOST_CHECK(s.playerLoc() == before);
133 }
134
135 BOOST_AUTO_TEST_SUITE_END()
136
137 // Border Interactions
138 BOOST_AUTO_TEST_SUITE(BorderTests)
139
140 BOOST_AUTO_TEST_CASE(WallBlocksAllDirections) {
141     std::string lvl =
142         "3 3\n"
143         "###\n"
144         "#@#\n"
145         "###\n";
146     SB::Sokoban s = makeGame(lvl);
147     sf::Vector2u pos = s.playerLoc();
148     s.movePlayer(SB::Direction::Up);
149     BOOST_CHECK(s.playerLoc() == pos);
150     s.movePlayer(SB::Direction::Down);
151     BOOST_CHECK(s.playerLoc() == pos);
152     s.movePlayer(SB::Direction::Left);
153     BOOST_CHECK(s.playerLoc() == pos);
154     s.movePlayer(SB::Direction::Right);
155     BOOST_CHECK(s.playerLoc() == pos);
156 }
157
158 BOOST_AUTO_TEST_SUITE_END()
159
160 // Victory Conditions
161 BOOST_AUTO_TEST_SUITE(VictoryTests)
162
```

```
163 BOOST_AUTO_TEST_CASE(StorageNotWonAtStart) {
164     std::string lvl =
165         "3 5\n"
166         "#####\n"
167         "#@Aa#\n"
168         "#####\n";
169     SB::Sokoban s = makeGame(lvl);
170     BOOST_CHECK(!s.isWon());
171 }
172
173 BOOST_AUTO_TEST_CASE(WonWithOneBoxAndMultipleTargets) {
174     std::string lvl =
175         "3 6\n"
176         "######\n"
177         "#@Aaa#\n"
178         "######\n";
179     SB::Sokoban s = makeGame(lvl);
180     s.movePlayer(SB::Direction::Right);
181     BOOST_CHECK(s.isWon());
182 }
183
184 BOOST_AUTO_TEST_CASE(NotWonFirstBoxOnStorageSecondIsNot) {
185     std::string lvl =
186         "3 8\n"
187         "########\n"
188         "#@Aa.Aa#\n"
189         "########\n";
190     SB::Sokoban s = makeGame(lvl);
191     s.movePlayer(SB::Direction::Right);
192     BOOST_CHECK(!s.isWon());
193 }
194
195 BOOST_AUTO_TEST_CASE(TwoBoxesBothOnStorageIsWon) {
196     std::string lvl =
197         "3 6\n"
198         "######\n"
199         "#1@Aa#\n"
200         "######\n";
201     SB::Sokoban s = makeGame(lvl);
202     s.movePlayer(SB::Direction::Right);
203     BOOST_CHECK(s.isWon());
204 }
205
206 BOOST_AUTO_TEST_SUITE_END()
```

5 PS4: AniPlayer

5.1 Assignment Overview

PS4 built a JSON-driven keyframe animation player. The program loads an animation description file, creates an SFML window sized to the declared dimensions, and plays back animated components with smooth tweening between keyframes. Supported component types are CircleComponent, RectangleComponent, ImageComponent, TextComponent, and CompositeComponent. Spacebar or P pauses and resumes playback; R restarts from the beginning. Extra credit: background music via sf::Music.

5.2 Key Concepts and Design Choices

Two design patterns were used on this assignment. The Factory pattern lives in a static Component::fromJson() method that reads the "shape" field and constructs the correct concrete subclass, returning a unique_ptr¡Component¿. AniPlayer never needs a type switch anywhere else in the codebase. The Composite pattern lets CompositeComponent hold a vector¡unique_ptr¡Component¿¿ of children and forward draw() calls to each one, passing sf::RenderStates down the chain so children automatically inherit the parent's accumulated transform without any manual arithmetic.

5.3 What I Learned

This was the most complex assignment of the semester. The Composite pattern showed how sf::RenderStates flows through a recursive draw() call to correctly position nested children without any global coordinate tracking. I also learned that using unique_ptr throughout meant zero manual delete calls and no memory leaks.

5.4 Evidence

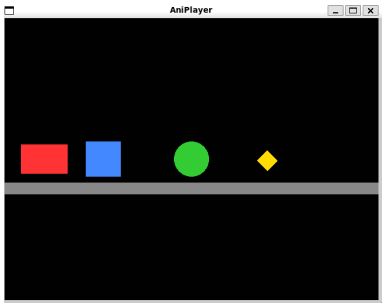


Figure 5: PS4: AniPlayer rendering a JSON keyframe animation mid-playback

5.5 Incomplete Features

Background music loaded without errors but produced no audio output in the WSL environment, which might be because WSL lacks an audio device by default. The code is correct and compiles cleanly.

5.6 Source Code

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Wpedantic -Werror -pedantic -g
SFML = -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio
BOOST_TEST = -lboost_unit_test_framework
LINT = cpplint
```

```
6
7 MAIN_SRC = main.cpp
8 LIB_SRC = AniPlayer.cpp \
9   Component.cpp \
10  KeyFrame.cpp \
11  CircleComponent.cpp \
12  RectangleComponent.cpp \
13  ImageComponent.cpp \
14  TextComponent.cpp \
15  CompositeComponent.cpp
16
17 TEST_SRC = test.cpp
18
19 MAIN_OBJ = $(MAIN_SRC:.cpp=.o)
20 LIB_OBJ = $(LIB_SRC:.cpp=.o)
21 TEST_OBJ = $(TEST_SRC:.cpp=.o)
22
23 ALL_SRC = $(MAIN_SRC) $(LIB_SRC)
24 ALL_HDR = AniPlayer.hpp Component.hpp KeyFrame.hpp \
25   CircleComponent.hpp RectangleComponent.hpp \
26   ImageComponent.hpp TextComponent.hpp CompositeComponent.hpp
27
28 .PHONY: all clean lint test
29
30 all: AniPlayer AniPlayer.a
31
32 AniPlayer: $(MAIN_OBJ) AniPlayer.a
33   $(CXX) $(CXXFLAGS) -o $@ $^ $(SFML)
34
35 AniPlayer.a: $(LIB_OBJ)
36   ar rcs $@ $^
37
38 test: $(TEST_OBJ) AniPlayer.a
39   $(CXX) $(CXXFLAGS) -o $@ $^ $(SFML) $(BOOST_TEST)
40   ./test
41
42 %.o: %.cpp
43   $(CXX) $(CXXFLAGS) -c -o $@ $<
44
45 lint:
46   $(LINT) --filter=-legal/copyright,-build/include_subdir \
47   $(ALL_SRC) $(ALL_HDR)
48
49 clean:
50   rm -f $(MAIN_OBJ) $(LIB_OBJ) $(TEST_OBJ) AniPlayer AniPlayer.a test
51
52 main.o: main.cpp AniPlayer.hpp Component.hpp KeyFrame.hpp
53 AniPlayer.o: AniPlayer.cpp AniPlayer.hpp Component.hpp KeyFrame.hpp
54 Component.o: Component.cpp Component.hpp KeyFrame.hpp \
55   CircleComponent.hpp RectangleComponent.hpp \
56   ImageComponent.hpp TextComponent.hpp CompositeComponent.hpp
57 KeyFrame.o: KeyFrame.cpp KeyFrame.hpp
58 CircleComponent.o: CircleComponent.cpp CircleComponent.hpp Component.hpp
59 RectangleComponent.o: RectangleComponent.cpp RectangleComponent.hpp Component.hpp
60 ImageComponent.o: ImageComponent.cpp ImageComponent.hpp Component.hpp
61 TextComponent.o: TextComponent.cpp TextComponent.hpp Component.hpp
62 CompositeComponent.o: CompositeComponent.cpp CompositeComponent.hpp Component.hpp
63 test.o: test.cpp KeyFrame.hpp Component.hpp
```

main.cpp

```
1 // Copyright 2026 Benny
2 #include <iostream>
3 #include <SFML/Graphics.hpp>
4 #include "AniPlayer.hpp"
5
```

```cpp
int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: " << argv[0] << " <video.json>" << std::endl;
        return 1;
    }

    AP::AniPlayer player(argv[1]);

    sf::Vector2u size = player.windowSize();
    sf::RenderWindow window(sf::VideoMode({size.x, size.y}), "AniPlayer");

    sf::Color bgColor(0x00, 0x00, 0x00, 0x00);

    sf::Clock clock;
    sf::Time elapsed = sf::Time::Zero;
    bool paused = false;
    bool ended = false;
    float videoMax = player.maxTime();

    while (window.isOpen()) {
        while (const std::optional<sf::Event> event = window.pollEvent()) {
            if (event->is<sf::Event::Closed>()) {
                window.close();
            }
            if (const auto* kp = event->getIf<sf::Event::KeyPressed>()) {
                if (kp->scancode == sf::Keyboard::Scan::Escape) {
                    window.close();
                } else if (kp->scancode == sf::Keyboard::Scan::R) {
                    // Restart
                    elapsed = sf::Time::Zero;
                    paused = false;
                    ended = false;
                    clock.restart();
                } else if (kp->scancode == sf::Keyboard::Scan::P ||
                           kp->scancode == sf::Keyboard::Scan::Space) {
                    // Toggle pause
                    if (!ended) {
                        paused = !paused;
                        if (paused) {
                            // Save accumulated time
                            elapsed += clock.getElapsedTime();
                        } else {
                            clock.restart();
                        }
                    }
                }
            }
        }

        if (!paused && !ended) {
            sf::Time currentTime = elapsed + clock.getElapsedTime();
            if (currentTime.asSeconds() >= videoMax) {
                currentTime = sf::seconds(videoMax);
                ended = true;
                elapsed = currentTime;
            }
            player.tween(currentTime);
        }

        window.clear(bgColor);
        window.draw(player);
        window.display();
    }

    return 0;
}
```

Aniplayer.hpp

```hpp
// Copyright 2026 Benny
#pragma once

#include <memory>
#include <string>
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>

#include "Component.hpp"

namespace AP {
class AniPlayer: public sf::Drawable {
 public:
    /**
     * Loads an AniPlayer containing the data from the given video
     * @param filename A .json file that contains data describing the video
     */
    explicit AniPlayer(const std::string& filename);

    /**
     * Advance the AniPlayer to the given time
     * @param time A given instance in time
     */
    void tween(sf::Time time);

    /**
     * Gets the viewport dimensions for this Drawable
     * @return The size (in pixels)
     */
    sf::Vector2u windowSize() const;

    /**
     * Returns the timestamp of the final keyframe across all components
     * @return Max time in seconds
     */
    float maxTime() const;

 protected:
    void draw(sf::RenderTarget& target, sf::RenderStates states) const override;

 private:
    unsigned int _width;
    unsigned int _height;
    std::unique_ptr<Component> _scene;
    sf::Music _bgm;
    bool _hasBgm;
};
} // namespace AP
```

Aniplayer.cpp

```cpp
// Copyright 2026 Benny
#include "AniPlayer.hpp"
#include <fstream>
#include <stdexcept>
#include <nlohmann/json.hpp>
using Json = nlohmann::json;

namespace AP {

AniPlayer::AniPlayer(const std::string& filename) : _width(0), _height(0), _hasBgm(false) {
    std::ifstream ifs(filename);
    if (!ifs.is_open()) {
```

```
        throw std::runtime_error("Could not open file: " + filename);
    }
    Json data = Json::parse(ifs);

    _width = data["width"].get<unsigned int>();
    _height = data["height"].get<unsigned int>();
    _scene = Component::fromJson(data["scene"]);

    if (data.contains("bgm")) {
        std::string bgmFile = data["bgm"].get<std::string>();
        if (_bgm.openFromFile(bgmFile)) {
            _hasBgm = true;
            _bgm.setLooping(true);
            _bgm.play();
        }
    }
}

void AniPlayer::tween(sf::Time time) {
    _scene->tween(time);
}

sf::Vector2u AniPlayer::windowSize() const {
    return sf::Vector2u(_width, _height);
}

float AniPlayer::maxTime() const {
    return _scene->maxTime();
}

void AniPlayer::draw(sf::RenderTarget& target, sf::RenderStates states) const {
    target.draw(*_scene, states);
}

} // namespace AP
```

KeyFrame.hpp

```
// Copyright 2026 Benny
#pragma once

#include <nlohmann/json.hpp>
#include <SFML/Graphics.hpp>
using Json = nlohmann::json;

namespace AP {
class KeyFrame {
 public:
    /**
     * Default constructs this KeyFrame
     */
    KeyFrame();

    /**
     * Constructs this KeyFrame by extracting data from a Json object
     * @param json A Json object containing KeyFrame data
     */
    explicit KeyFrame(const Json& json);

    /**
     * Applies the transform information in this keyframe to the given Transform.
     * @param state A Transform object that will be modified
     */
    void transform(sf::Transform& state) const;

    /**
```

```
    * Returns the timestamp of this KeyFrame
    * @return The time t in seconds
    */
   float t() const;

   /**
    * Linearly interpolates between two KeyFrames at time t (in seconds).
    * @param a The earlier KeyFrame
    * @param b The later KeyFrame
    * @param t The desired time in seconds
    * @return An interpolated KeyFrame at time t
    */
   static KeyFrame tweenBetween(const KeyFrame& a, const KeyFrame& b, float t);

private:
   float _t;
   float _x;
   float _y;
   float _scaleX;
   float _scaleY;
   float _theta;
};
} // namespace AP
```

KeyFrame.cpp

```
// Copyright 2026 Benny
#include "KeyFrame.hpp"

namespace AP {

KeyFrame::KeyFrame() : _t(0), _x(0), _y(0), _scaleX(1), _scaleY(1), _theta(0) {}

KeyFrame::KeyFrame(const Json& json) {
   _t = json["t"].get<float>();
   _x = json.value("x", 0.0f);
   _y = json.value("y", 0.0f);
   _scaleX = json.value("scale-x", 1.0f);
   _scaleY = json.value("scale-y", 1.0f);
   _theta = json.value("theta", 0.0f);
}

void KeyFrame::transform(sf::Transform& state) const {
   state.translate(sf::Vector2f(_x, _y));
   state.rotate(sf::degrees(_theta));
   state.scale(sf::Vector2f(_scaleX, _scaleY));
}

float KeyFrame::t() const {
   return _t;
}

KeyFrame KeyFrame::tweenBetween(const KeyFrame& a, const KeyFrame& b, float t) {
   float dt = b._t - a._t;
   float alpha = (dt == 0.0f) ? 1.0f : (t - a._t) / dt;

   KeyFrame result;
   result._t = t;
   result._x = a._x + alpha * (b._x - a._x);
   result._y = a._y + alpha * (b._y - a._y);
   result._scaleX = a._scaleX + alpha * (b._scaleX - a._scaleX);
   result._scaleY = a._scaleY + alpha * (b._scaleY - a._scaleY);
   result._theta = a._theta + alpha * (b._theta - a._theta);
   return result;
}

```

```
} // namespace AP
```

Component.hpp

```hpp
// Copyright 2026 Benny
#pragma once

#include <memory>
#include <vector>
#include <nlohmann/json.hpp>
using Json = nlohmann::json;
#include <SFML/Graphics.hpp>

#include "KeyFrame.hpp"

namespace AP {
class Component: public sf::Drawable {
 public:
    /**
     * Constructs a Component from the Json object
     * @param data A Json object containing Component data
     * @return A new Component
     */
    static std::unique_ptr<Component> fromJson(const Json& data); // Factory

    /**
     * Advances this component to the state appropriate to the given time
     * @param time The Time to move to
     */
    virtual void tween(sf::Time time);

    /**
     * Returns the timestamp of the final keyframe (0 if none)
     * @return Max time in seconds
     */
    virtual float maxTime() const;

 protected:
    /**
     * Initializes base class components from the Json object
     * @param data A Json object containing Component data
     */
    explicit Component(const Json& data); // Abstract class

    virtual void draw(sf::RenderTarget& window, sf::RenderStates states) const = 0;

    /**
     * Gets the interpolated KeyFrame for the current time set via tween()
     * @return The interpolated KeyFrame
     */
    KeyFrame currFrame() const;

    /**
     * Gets this Component's color
     * @return The color
     */
    sf::Color color() const;

 private:
    sf::Color _color;
    std::vector<KeyFrame> _keyframes;
    sf::Time _currTime;
};
} // namespace AP
```

Component.cpp

```cpp
// Copyright 2026 Benny
#include "Component.hpp"
#include <stdexcept>
#include <string>

#include "CircleComponent.hpp"
#include "RectangleComponent.hpp"
#include "ImageComponent.hpp"
#include "TextComponent.hpp"
#include "CompositeComponent.hpp"

namespace AP {

static sf::Color parseColor(const std::string& hex) {
    std::string h = hex;
    if (!h.empty() && h[0] == '#') h = h.substr(1);
    if (h.size() == 6) h += "FF";
    if (h.size() != 8) return sf::Color::White;
    unsigned int val = std::stoul(h, nullptr, 16);
    uint8_t r = (val >> 24) & 0xFF;
    uint8_t g = (val >> 16) & 0xFF;
    uint8_t b = (val >> 8) & 0xFF;
    uint8_t a = (val) & 0xFF;
    return sf::Color(r, g, b, a);
}

Component::Component(const Json& data) : _currTime(sf::Time::Zero) {
    std::string colorStr = data.value("color", "FFFFFFFF");
    _color = parseColor(colorStr);

    if (data.contains("keyframes")) {
        for (const Json& kf : data["keyframes"]) {
            _keyframes.emplace_back(kf);
        }
    }
}

void Component::tween(sf::Time time) {
    _currTime = time;
}

KeyFrame Component::currFrame() const {
    if (_keyframes.empty()) return KeyFrame();

    float t = _currTime.asSeconds();

    // Before first keyframe: hold first
    if (t <= _keyframes.front().t()) return _keyframes.front();

    // After last keyframe: hold last
    if (t >= _keyframes.back().t()) return _keyframes.back();

    // Find surrounding keyframes and interpolate
    for (size_t i = 0; i + 1 < _keyframes.size(); ++i) {
        const KeyFrame& a = _keyframes[i];
        const KeyFrame& b = _keyframes[i + 1];
        if (t >= a.t() && t <= b.t()) {
            return KeyFrame::tweenBetween(a, b, t);
        }
    }

    return _keyframes.back();
}

float Component::maxTime() const {
    if (_keyframes.empty()) return 0.0f;
```

```
    return _keyframes.back().t();
}

sf::Color Component::color() const {
    return _color;
}

std::unique_ptr<Component> Component::fromJson(const Json& data) {
    std::string shape = data["shape"];
    if (shape == "circle") {
        return std::make_unique<CircleComponent>(data);
    } else if (shape == "rect") {
        return std::make_unique<RectangleComponent>(data);
    } else if (shape == "image") {
        return std::make_unique<ImageComponent>(data);
    } else if (shape == "text") {
        return std::make_unique<TextComponent>(data);
    } else if (shape == "composite") {
        return std::make_unique<CompositeComponent>(data);
    } else {
        throw std::runtime_error("Unknown component shape: " + shape);
    }
}

} // namespace AP
```

CompositeComponent.hpp

```
// Copyright 2026 Benny
#pragma once

#include <memory>
#include <vector>

#include "Component.hpp"

namespace AP {
class CompositeComponent : public Component {
 public:
    explicit CompositeComponent(const Json& data);
    void tween(sf::Time time) override;
    float maxTime() const override;
 protected:
    void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
 private:
    std::vector<std::unique_ptr<Component>> _children;
};
} // namespace AP
```

CompositeComponent.cpp

```
// Copyright 2026 Benny
#include "CompositeComponent.hpp"
#include <algorithm>

namespace AP {

CompositeComponent::CompositeComponent(const Json& data) : Component(data) {
    if (data.contains("children")) {
        for (const Json& child : data["children"]) {
            _children.push_back(Component::fromJson(child));
        }
    }
}
```

```
14
15 void CompositeComponent::tween(sf::Time time) {
16     Component::tween(time);
17     for (auto& child : _children) {
18         child->tween(time);
19     }
20 }
21
22 float CompositeComponent::maxTime() const {
23     float m = Component::maxTime();
24     for (const auto& child : _children) {
25         m = std::max(m, child->maxTime());
26     }
27     return m;
28 }
29
30 void CompositeComponent::draw(sf::RenderTarget& window, sf::RenderStates states) const {
31     KeyFrame kf = currFrame();
32     kf.transform(states.transform);
33     for (const auto& child : _children) {
34         window.draw(*child, states);
35     }
36 }
37
38 } // namespace AP
```

CircleComponent.hpp

```
1  // Copyright 2026 Benny
2  #pragma once
3  #include "Component.hpp"
4
5  namespace AP {
6  class CircleComponent : public Component {
7   public:
8      explicit CircleComponent(const Json& data);
9   protected:
10     void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
11  private:
12     float _radius;
13     size_t _sides;
14 };
15 } // namespace AP
```

CircleComponent.cpp

```
1  // Copyright 2026 Benny
2  #include "CircleComponent.hpp"
3
4  namespace AP {
5
6  CircleComponent::CircleComponent(const Json& data) : Component(data) {
7      _radius = data.value("radius", 0.0f);
8      _sides = data.value("sides", static_cast<size_t>(30));
9  }
10
11 void CircleComponent::draw(sf::RenderTarget& window, sf::RenderStates states) const {
12     KeyFrame kf = currFrame();
13     kf.transform(states.transform);
14
15     sf::CircleShape circle(_radius, _sides);
16     circle.setFillColor(color());
17     circle.setOrigin(sf::Vector2f(_radius, _radius));
18     window.draw(circle, states);
```

```
}

} // namespace AP
```

RectangleComponent.hpp

```
// Copyright 2026 Benny
#pragma once
#include "Component.hpp"

namespace AP {
class RectangleComponent : public Component {
 public:
    explicit RectangleComponent(const Json& data);
 protected:
    void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
 private:
    float _rx; // local offset x
    float _ry; // local offset y
    float _width;
    float _height;
};
} // namespace AP
```

RectangleComponent.cpp

```
// Copyright 2026 Benny
#include "RectangleComponent.hpp"

namespace AP {

RectangleComponent::RectangleComponent(const Json& data) : Component(data) {
    _rx = data.value("x", 0.0f);
    _ry = data.value("y", 0.0f);
    _width = data.value("width", 0.0f);
    _height = data.value("height", 0.0f);
}

void RectangleComponent::draw(sf::RenderTarget& window, sf::RenderStates states) const {
    KeyFrame kf = currFrame();
    kf.transform(states.transform);

    sf::RectangleShape rect(sf::Vector2f(_width, _height));
    rect.setFillColor(color());
    rect.setPosition(sf::Vector2f(_rx, _ry));
    window.draw(rect, states);
}

} // namespace AP
```

ImageComponent.hpp

```
// Copyright 2026 Benny
#pragma once
#include <string>
#include "Component.hpp"
#include <SFML/Graphics.hpp>

namespace AP {
class ImageComponent : public Component {
 public:
```

```
    explicit ImageComponent(const Json& data);
protected:
    void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
private:
    std::string _file;
    sf::Texture _texture;
};
} // namespace AP
```

ImageComponent.cpp

```
// Copyright 2026 Benny
#include "ImageComponent.hpp"
#include <stdexcept>

namespace AP {

ImageComponent::ImageComponent(const Json& data) : Component(data) {
    _file = data["file"].get<std::string>();
    if (!_texture.loadFromFile(_file)) {
        throw std::runtime_error("Could not load image: " + _file);
    }
}

void ImageComponent::draw(sf::RenderTarget& window, sf::RenderStates states) const {
    KeyFrame kf = currFrame();
    kf.transform(states.transform);

    sf::Sprite sprite(_texture);
    sf::FloatRect bounds = sprite.getLocalBounds();
    sprite.setOrigin(sf::Vector2f(bounds.size.x / 2.0f, bounds.size.y / 2.0f));
    sprite.setColor(color());
    window.draw(sprite, states);
}

} // namespace AP
```

TextComponent.hpp

```
// Copyright 2026 Benny
#pragma once
#include <string>
#include "Component.hpp"
#include <SFML/Graphics.hpp>

namespace AP {
class TextComponent : public Component {
public:
    explicit TextComponent(const Json& data);
protected:
    void draw(sf::RenderTarget& window, sf::RenderStates states) const override;
private:
    std::string _text;
    std::string _fontFile;
    sf::Font _font;
};
} // namespace AP
```

TextComponent.cpp

```cpp
// Copyright 2026 Benny
#include "TextComponent.hpp"
#include <stdexcept>

namespace AP {

TextComponent::TextComponent(const Json& data) : Component(data) {
    _text = data.value("text", std::string(""));
    _fontFile = data["font"].get<std::string>();
    if (!_font.openFromFile(_fontFile)) {
        throw std::runtime_error("Could not load font: " + _fontFile);
    }
}

void TextComponent::draw(sf::RenderTarget& window, sf::RenderStates states) const {
    KeyFrame kf = currFrame();
    kf.transform(states.transform);

    sf::Text text(_font, _text);
    text.setFillColor(color());
    window.draw(text, states);
}

} // namespace AP
```

test.cpp

```cpp
// Copyright 2026 Benny
#define BOOST_TEST_DYN_LINK
#define BOOST_TEST_MODULE Test
#include <boost/test/unit_test.hpp>

#include "KeyFrame.hpp"
#include "Component.hpp"

#include <nlohmann/json.hpp>
#include <SFML/Graphics.hpp>

using Json = nlohmann::json;

static Json makeJson(float t, float x, float y,
                     float scaleX = 1.0f, float scaleY = 1.0f, float theta = 0.0f) {
    return Json{{"t", t}, {"x", x}, {"y", y},
                {"scale-x", scaleX}, {"scale-y", scaleY}, {"theta", theta}};
}

static Json makeRectJson(std::vector<Json> keyframes, float w = 50, float h = 50) {
    return Json{{"shape", "rect"}, {"width", w}, {"height", h},
                {"color", "FF0000FF"}, {"keyframes", keyframes}};
}

static bool anyRed(AP::Component& comp) {
    sf::RenderTexture rt({500, 500});
    rt.clear(sf::Color::Black);
    rt.draw(comp, sf::RenderStates::Default);
    rt.display();
    auto img = rt.getTexture().copyToImage();
    for (unsigned int y = 0; y < 500; y++)
        for (unsigned int x = 0; x < 500; x++)
            if (img.getPixel({x, y}).r > 100) return true;
    return false;
}

static sf::Color getPixel(AP::Component& comp, unsigned int x, unsigned int y) {
    sf::RenderTexture rt({500, 500});
```

```
39      rt.clear(sf::Color::Black);
40      rt.draw(comp, sf::RenderStates::Default);
41      rt.display();
42      return rt.getTexture().copyToImage().getPixel({x, y});
43  }
44
45  BOOST_AUTO_TEST_SUITE(KeyFrameSuite)
46
47  BOOST_AUTO_TEST_CASE(TweeningCompleteness) {
48      Json j = makeRectJson({makeJson(0, 0, 0), makeJson(2, 200, 0)});
49      auto comp = AP::Component::fromJson(j);
50
51      comp->tween(sf::seconds(1.0f));
52      sf::Color mid = getPixel(*comp, 125, 25);
53      sf::Color origin = getPixel(*comp, 25, 25);
54
55      BOOST_CHECK(mid.r > 100);
56      BOOST_CHECK(origin.r < 50);
57  }
58
59  BOOST_AUTO_TEST_CASE(TweeningDirection) {
60      Json j = makeRectJson({makeJson(0, 0, 0), makeJson(4, 400, 0)});
61      auto comp = AP::Component::fromJson(j);
62
63      comp->tween(sf::seconds(1.0f));
64      sf::Color early = getPixel(*comp, 325, 25);
65
66      comp->tween(sf::seconds(3.0f));
67      sf::Color late = getPixel(*comp, 325, 25);
68
69      BOOST_CHECK(early.r < 50);
70      BOOST_CHECK(late.r > 100);
71  }
72
73  BOOST_AUTO_TEST_CASE(FixedKeyframe) {
74      Json j = makeRectJson({makeJson(2, 0, 0), makeJson(2, 200, 0)});
75      auto comp = AP::Component::fromJson(j);
76
77      comp->tween(sf::seconds(2.0f));
78      BOOST_CHECK(anyRed(*comp));
79  }
80
81  BOOST_AUTO_TEST_CASE(OnlyKeyframes) {
82      Json j = makeRectJson({makeJson(5, 100, 0)});
83      auto comp = AP::Component::fromJson(j);
84
85      comp->tween(sf::seconds(0.0f));
86      BOOST_CHECK(anyRed(*comp));
87
88      comp->tween(sf::seconds(100.0f));
89      BOOST_CHECK(anyRed(*comp));
90  }
91
92  BOOST_AUTO_TEST_CASE(TemporalBoundary) {
93      Json j = makeRectJson({makeJson(1, 0, 0), makeJson(3, 200, 0)});
94      auto comp = AP::Component::fromJson(j);
95
96      comp->tween(sf::seconds(0.0f));
97      sf::Color atStart = getPixel(*comp, 25, 25);
98
99      comp->tween(sf::seconds(10.0f));
100     sf::Color atEnd = getPixel(*comp, 225, 25);
101
102     BOOST_CHECK(atStart.r > 100);
103     BOOST_CHECK(atEnd.r > 100);
104 }
105
106 BOOST_AUTO_TEST_SUITE_END()
```

6 PS5: DNA Alignment

6.1 Assignment Overview

This project calculates the optimal alignment between two given strings using dynamic programming. It determines the minimal edit distance, the total cost to transform one string into another through matches, mismatches, and gaps. The solution builds a matrix of edit costs and traces back to reconstruct the best alignment. The result shows which character aligns, mismatches, or needs a gap. The output displays both the edit distance and a detailed alignment with the cost annotations as well as the time it takes to complete the task, where a mismatch costs 1, and a gap costs 2.

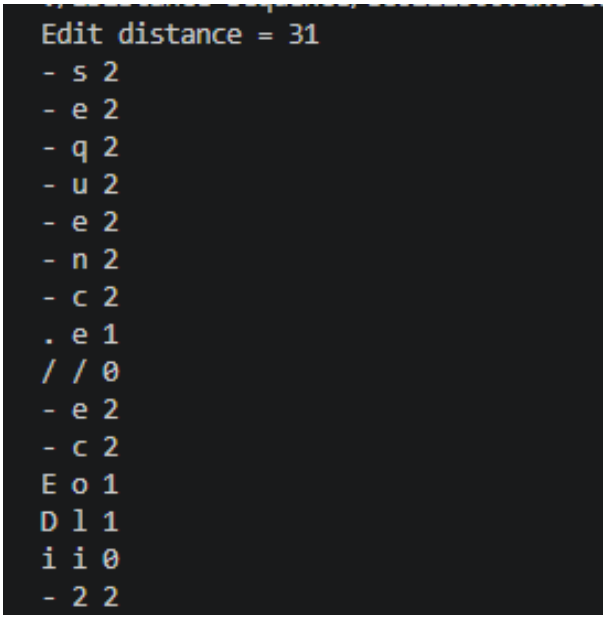
6.2 Key Concepts and Design Choices

The design for this project was done using what was provided during class. The implementation uses dynamic programming using a 2D vector matrix. The algorithm fills the matrix from top left to bottom right. min3() helper handles the three-way comparison since std::min only takes two values. For the alignment the process is the opposite and spits it back out in reverse, checking the cells again and asigning penaltys using the penalty function. Clock and time from sfml was used to get the elapsed time.

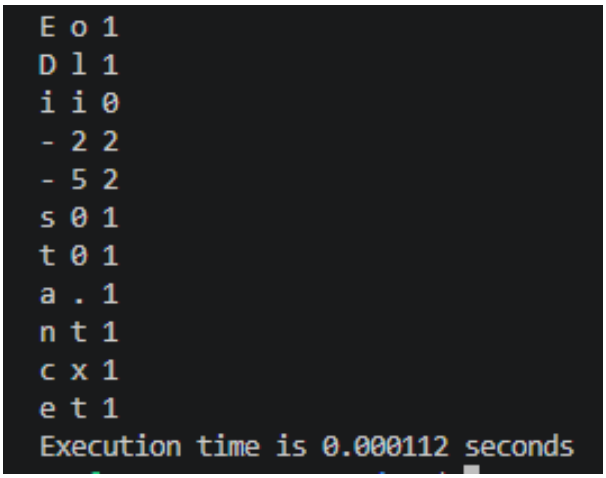
6.3 What I Learned

Dynamic programming provides an efficient alternative to inefficient recursion by storing computed subproblems in a table. This approach avoids redundant calculations. The technique demonstrates how careful data structure design can dramatically improve speed and effiency. C++ has a min function that can be included using the algorithm library, no need to make calculations..

6.4 Evidence



(a) Alignment output



(b) Timing results

Figure 6: PS5: ecoli2500.txt results

6.5 Incomplete Features

The largest two test cases (ecoli50000.txt and ecoli100000.txt) crashed with out-of-memory errors on both computers because the table exceeds available RAM at those sizes, and getting the right alignment was a challenge. This took most of the time for this project. No known incomplete features.

6.6 Source Code

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Wpedantic -Werror -pedantic -g
LIB = -lsfml-system -lboost_unit_test_framework

all: EDistance test EDistance.a

EDistance: main.o EDistance.o
  $(CXX) $(CXXFLAGS) -o EDistance main.o EDistance.o $(LIB)

test: test.o EDistance.o
  $(CXX) $(CXXFLAGS) -o test test.o EDistance.o $(LIB)

EDistance.a: EDistance.o
  ar rcs EDistance.a EDistance.o

main.o: main.cpp EDistance.hpp
  $(CXX) $(CXXFLAGS) -c main.cpp

EDistance.o: EDistance.cpp EDistance.hpp
  $(CXX) $(CXXFLAGS) -c EDistance.cpp

test.o: test.cpp EDistance.hpp
  $(CXX) $(CXXFLAGS) -c test.cpp

lint:
  cpplint --filter=-legal/copyright EDistance.hpp EDistance.cpp main.cpp test.cpp

clean:
  rm -f *.o EDistance test EDistance.a
```

main.cpp

```
// Copyright 2026 Benny
#include <iostream>
#include <string>
#include <SFML/System.hpp>
#include "EDistance.hpp"

int main() {
    std::string s1, s2;
    std::cin >> s1 >> s2;

    if (!s1.empty() && s1.back() == '\r') s1.pop_back();
    if (!s2.empty() && s2.back() == '\r') s2.pop_back();

    sf::Clock clock;

    EDistance ed(s1, s2);
    int dist = ed.optDistance();

    std::cout << "Edit distance = " << dist << "\n";
    std::cout << ed.alignment();

    sf::Time t = clock.getElapsedTime();
```

```
    std::cout << "Execution time is " << t.asSeconds() << " seconds" << std::endl;

    return 0;
}
```

EDistance.hpp

```
// Copyright 2026 Benny
#pragma once

#include <string>
#include <vector>

class EDistance {
 public:
    EDistance(const std::string& s1, const std::string& s2);
    ~EDistance();

    static int penalty(char a, char b);
    static int min3(int a, int b, int c);

    int optDistance();
    std::string alignment();

 private:
    std::string _x, _y;
    int _m, _n;
    int* _opt;

    int& at(int i, int j) { return _opt[i * (_n + 1) + j]; }
};
```

EDistance.cpp

```
// Copyright 2026 Benny
#include "EDistance.hpp"
#include <stdexcept>
#include <sstream>
#include <vector>

EDistance::EDistance(const std::string& s1, const std::string& s2)
    : _x(s1), _y(s2), _m(s1.size()), _n(s2.size()) {
    _opt = new int[(_m + 1) * (_n + 1)];
}

EDistance::~EDistance() {
    delete[] _opt;
}

int EDistance::penalty(char a, char b) {
    return (a == b) ? 0 : 1;
}

int EDistance::min3(int a, int b, int c) {
    if (a <= b && a <= c) return a;
    if (b <= c) return b;
    return c;
}

int EDistance::optDistance() {
    for (int j = 0; j <= _n; j++)
        at(0, j) = j * 2;
    for (int i = 0; i <= _m; i++)
        at(i, 0) = i * 2;
```

```
31
32      // fill top-to-bottom, left to right
33      for (int i = 1; i <= _m; i++) {
34          for (int j = 1; j <= _n; j++) {
35              int match = at(i - 1, j - 1) + penalty(_x[i - 1], _y[j - 1]);
36              int del = at(i - 1, j) + 2;
37              int ins = at(i, j - 1) + 2;
38              at(i, j) = min3(match, del, ins);
39          }
40      }
41      return at(_m, _n);
42  }
43
44  std::string EDistance::alignment() {
45      std::ostringstream oss;
46      int i = _m, j = _n;
47      while (i > 0 || j > 0) {
48          if (i > 0 && j > 0) {
49              int p = penalty(_x[i - 1], _y[j - 1]);
50              if (at(i, j) == at(i - 1, j - 1) + p) {
51                  oss << _x[i - 1] << ' ' << _y[j - 1] << ' ' << p << '\n';
52                  i--; j--;
53                  continue;
54              }
55          }
56          if (i > 0 && at(i, j) == at(i - 1, j) + 2) {
57              oss << _x[i - 1] << " - 2\n";
58              i--;
59          } else {
60              oss << "- " << _y[j - 1] << " 2\n";
61              j--;
62          }
63      }
64      std::string s = oss.str();
65      std::string result;
66      std::vector<std::string> lines;
67      std::istringstream ss(s);
68      std::string line;
69      while (std::getline(ss, line)) lines.push_back(line);
70      for (int k = lines.size() - 1; k >= 0; k--)
71          result += lines[k] + '\n';
72      return result;
73  }
```

test.cpp

```
1  // Copyright 2026 Benny
2  #define BOOST_TEST_DYN_LINK
3  #define BOOST_TEST_MODULE EDistanceTest
4  #include <string>
5  #include <sstream>
6  #include <vector>
7  #include <boost/test/unit_test.hpp>
8  #include "EDistance.hpp"
9
10 BOOST_AUTO_TEST_CASE(min3_basic) {
11     // All permuations
12     BOOST_REQUIRE_EQUAL(EDistance::min3(1, 2, 3), 1);
13     BOOST_REQUIRE_EQUAL(EDistance::min3(1, 3, 2), 1);
14     BOOST_REQUIRE_EQUAL(EDistance::min3(2, 1, 3), 1);
15     BOOST_REQUIRE_EQUAL(EDistance::min3(3, 1, 2), 1);
16     BOOST_REQUIRE_EQUAL(EDistance::min3(2, 3, 1), 1);
17     BOOST_REQUIRE_EQUAL(EDistance::min3(3, 2, 1), 1);
18     // Equal values
19     BOOST_REQUIRE_EQUAL(EDistance::min3(2, 2, 2), 2);
20     BOOST_REQUIRE_EQUAL(EDistance::min3(1, 1, 2), 1);
```

```
21      BOOST_REQUIRE_EQUAL(EDistance::min3(2, 1, 1), 1);
22      BOOST_REQUIRE_EQUAL(EDistance::min3(1, 2, 1), 1);
23      // b min
24      BOOST_REQUIRE_EQUAL(EDistance::min3(9, 1, 9), 1);
25      BOOST_REQUIRE_EQUAL(EDistance::min3(7, 2, 5), 2);
26      BOOST_REQUIRE_EQUAL(EDistance::min3(4, 0, 3), 0);
27      // c min
28      BOOST_REQUIRE_EQUAL(EDistance::min3(9, 9, 1), 1);
29      BOOST_REQUIRE_EQUAL(EDistance::min3(5, 7, 2), 2);
30      BOOST_REQUIRE_EQUAL(EDistance::min3(3, 4, 0), 0);
31      // a min
32      BOOST_REQUIRE_EQUAL(EDistance::min3(1, 9, 9), 1);
33      BOOST_REQUIRE_EQUAL(EDistance::min3(2, 5, 7), 2);
34      BOOST_REQUIRE_EQUAL(EDistance::min3(0, 3, 4), 0);
35  }
36
37  BOOST_AUTO_TEST_CASE(cost_function) {
38      BOOST_REQUIRE_EQUAL(EDistance::penalty('A', 'A'), 0);
39      BOOST_REQUIRE_EQUAL(EDistance::penalty('C', 'C'), 0);
40      BOOST_REQUIRE_EQUAL(EDistance::penalty('A', 'T'), 1);
41      BOOST_REQUIRE_EQUAL(EDistance::penalty('G', 'C'), 1);
42  }
43
44  BOOST_AUTO_TEST_CASE(basic_string_formatting) {
45      EDistance ed("AACAGTTACC", "TAAGGTCA");
46      ed.optDistance();
47      std::string a = ed.alignment();
48
49      std::istringstream ss(a);
50      std::vector<std::string> lines;
51      std::string line;
52      while (std::getline(ss, line)) {
53          if (!line.empty()) lines.push_back(line);
54      }
55
56      BOOST_REQUIRE_EQUAL(lines.size(), 10);
57
58      for (const auto& l : lines) {
59          BOOST_REQUIRE_EQUAL(l.size(), 5);
60          BOOST_REQUIRE_EQUAL(l[1], ' ');
61          BOOST_REQUIRE_EQUAL(l[3], ' ');
62      }
63
64      BOOST_REQUIRE_EQUAL(lines.front(), "A T 1");
65      BOOST_REQUIRE_EQUAL(lines.back(), "C A 1");
66
67      std::string xcol;
68      for (const auto& l : lines) if (l[0] != '-') xcol += l[0];
69      BOOST_REQUIRE_EQUAL(xcol, "AACAGTTACC");
70  }
71
72  BOOST_AUTO_TEST_CASE(alignment_head_coverage) {
73      EDistance ed_exact("ACG", "ACG");
74      ed_exact.optDistance();
75      std::string a = ed_exact.alignment();
76
77      std::istringstream ss(a);
78      std::vector<std::string> lines;
79      std::string line;
80      while (std::getline(ss, line)) {
81          if (!line.empty()) lines.push_back(line);
82      }
83
84      BOOST_REQUIRE_EQUAL(lines.size(), 3);
85      BOOST_REQUIRE_EQUAL(lines[0], "A A 0");
86      BOOST_REQUIRE_EQUAL(lines[1], "C C 0");
87      BOOST_REQUIRE_EQUAL(lines[2], "G G 0");
88
```

```
    EDistance ed_gap("TT", "ATT");
    ed_gap.optDistance();
    std::string a2 = ed_gap.alignment();

    std::istringstream ss2(a2);
    std::vector<std::string> lines2;
    while (std::getline(ss2, line)) {
        if (!line.empty()) lines2.push_back(line);
    }

    BOOST_REQUIRE_EQUAL(lines2.size(), 3);
    BOOST_REQUIRE_EQUAL(lines2[0], "- A 2");
    BOOST_REQUIRE_EQUAL(lines2[1], "T T 0");
    BOOST_REQUIRE_EQUAL(lines2[2], "T T 0");

    EDistance ed_mismatch("CA", "TA");
    ed_mismatch.optDistance();
    std::string a3 = ed_mismatch.alignment();

    std::istringstream ss3(a3);
    std::vector<std::string> lines3;
    while (std::getline(ss3, line)) {
        if (!line.empty()) lines3.push_back(line);
    }

    BOOST_REQUIRE_EQUAL(lines3.size(), 2);
    BOOST_REQUIRE_EQUAL(lines3[0], "C T 1");
    BOOST_REQUIRE_EQUAL(lines3[1], "A A 0");
}

BOOST_AUTO_TEST_CASE(improper_string_matching) {
    EDistance ed("AACAGTTACC", "TAAGGTCA");
    BOOST_REQUIRE_EQUAL(ed.optDistance(), 7);

    EDistance ed2("TAAGGTCA", "AACAGTTACC");
    BOOST_REQUIRE_EQUAL(ed2.optDistance(), 7);

    EDistance ed3("AAAA", "TTTT");
    BOOST_REQUIRE_EQUAL(ed3.optDistance(), 4);

    EDistance ed4("ACG", "ACGTT");
    BOOST_REQUIRE_EQUAL(ed4.optDistance(), 4);
    ed4.optDistance();
    std::string a4 = ed4.alignment();
    BOOST_REQUIRE(a4.find("- T 2") != std::string::npos);

    EDistance ed5("ACGTT", "ACG");
    BOOST_REQUIRE_EQUAL(ed5.optDistance(), 4);
    ed5.optDistance();
    std::string a5 = ed5.alignment();
    BOOST_REQUIRE(a5.find("T - 2") != std::string::npos);
}
```

7 PS6: RandWriter

7.1 Assignment Overview

PS6 implemented an order-k Markov model to analyze patterns and text generation. The program reads an input text, learns which characters follow each k-length substring (k-gram), and generates new text that mimics the original pattern and style using probability.

7.2 Key Concepts and Design Choices

The core data structure is std::map¡std::string, std::map¡char, int¿¿. The inner map counts how many times each character follows it in the source text. kRand() adds all follower counts for a k-gram, picks a random integer using std::mt19937, then walks the inner map, accumulating counts until the running sum meets the roll. A lambda [](const std::pair¡const char, int¿ entry) is passed to std::for_each in kRand() to satisfy the required lambda-as-parameter technique. generate() repeatedly calls kRand() and advances the current k-gram by dropping its first character and appending the new one.

7.3 What I Learned

A Markov model captures local statistical structure without any understanding of meaning: at order 2, the output looks like random noise, but by order 7–8 on English text it produces convincing-looking words and occasional real phrases. I also Learned that std::mt19937 is a better randomizer than rand().

7.4 Evidence

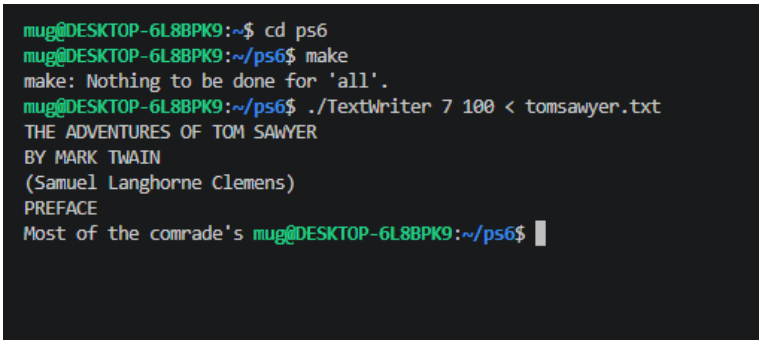


Figure 7: PS6: generated text output from TextWriter at varying orders

7.5 Incomplete Features

No known incomplete features.

7.6 Source Code

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Wpedantic -Werror -pedantic -g
LDFLAGS =
BOOSTFLAGS = -lboost_unit_test_framework

all: TextWriter TextWriter.a test

TextWriter.a: RandWriter.o
```

```
  ar rcs $@ $^

RandWriter.o: RandWriter.cpp RandWriter.hpp
  $(CXX) $(CXXFLAGS) -c $< -o $@

TextWriter: TextWriter.o TextWriter.a
  $(CXX) $(CXXFLAGS) $^ -o $@ $(LDFLAGS)

TextWriter.o: TextWriter.cpp RandWriter.hpp
  $(CXX) $(CXXFLAGS) -c $< -o $@

test: test.o TextWriter.a
  $(CXX) $(CXXFLAGS) $^ -o $@ $(BOOSTFLAGS)

test.o: test.cpp RandWriter.hpp
  $(CXX) $(CXXFLAGS) -c $< -o $@

lint:
  cpplint --filter=-legal/copyright RandWriter.cpp RandWriter.hpp TextWriter.cpp test.cpp

clean:
  rm -f *.o TextWriter TextWriter.a test
```

TextWriter.cpp

```
// Copyright 2026 Benny
#include <iostream>
#include <string>
#include <stdexcept>
#include "RandWriter.hpp"

int main(int argc, char* argv[]) {
    if (argc != 3) {
        std::cerr << "Usage: " << argv[0] << " k L\n";
        return 1;
    }

    size_t k = static_cast<size_t>(std::stoul(argv[1]));
    size_t L = static_cast<size_t>(std::stoul(argv[2]));

    std::string text(std::istreambuf_iterator<char>(std::cin),
    std::istreambuf_iterator<char>());

    try {
        RandWriter rw(text, k);
        std::string seed = (k == 0) ? "" : text.substr(0, k);
        std::string output = rw.generate(seed, L);
        std::cout << output;
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 1;
    }

    return 0;
}
```

RandWriter.hpp

```
// Copyright 2026 Benny
#pragma once
#include <string>
#include <map>
#include <random>
#include <stdexcept>
```

```
7
8 class RandWriter {
9  public:
10     // Create a Markov model of order k from given text
11     // Assume that text has length at least k.
12     RandWriter(const std::string& str, size_t k);
13
14     size_t orderK() const; // Order k of Markov model
15
16     // Number of occurences of kgram in text
17     // Throw an exception if kgram is not length k
18     int freq(const std::string& kgram) const;
19     // Number of times that character c follows kgram
20     // if order=0, return num of times that char c appears
21     // (throw an exception if kgram is not of length k)
22     int freq(const std::string& kgram, char c) const;
23
24     // Random character following given kgram
25     // (throw an exception if kgram is not of length k)
26     // (throw an exception if no such kgram)
27     char kRand(const std::string& kgram);
28     // Generate a string of length L characters by simulating a trajectory
29     // through the corresponding Markov chain. The first k characters of
30     // the newly generated string should be the argument kgram.
31     // Throw an exception if kgram is not of length k.
32     // Assume that L is at least k
33     std::string generate(const std::string& kgram, size_t l);
34
35     friend std::ostream& operator<<(std::ostream& os, const RandWriter& rw);
36
37  private:
38 // Private member variables go here
39     size_t _k;
40     std::string _alphabet;
41     std::map<std::string, std::map<char, int>> _table;
42     std::mt19937 _rng;
43 };
```

RandWriter.cpp

```
1 // Copyright 2026 Benny
2 #include "RandWriter.hpp"
3 #include <algorithm>
4 #include <iostream>
5 #include <stdexcept>
6 #include <string>
7 #include <utility>
8 #include <chrono>
9
10 RandWriter::RandWriter(const std::string& text, size_t k)
11     : _k(k), _rng(std::chrono::steady_clock::now().time_since_epoch().count()) {
12     if (text.size() < k) {
13         throw std::invalid_argument("Text length must be at least k");
14     }
15
16     std::string sorted = text;
17     std::sort(sorted.begin(), sorted.end());
18     _alphabet = std::string(sorted.begin(),
19     std::unique(sorted.begin(), sorted.end()));
20
21     if (k == 0) {
22         for (char c : text) {
23             _table[""][c]++;
24         }
25         return;
26     }
```

```
27
28      std::string circular = text + text.substr(0, k);
29      for (size_t i = 0; i < text.size(); i++) {
30          std::string kgram = circular.substr(i, k);
31          char next = circular[i + k];
32          _table[kgram][next]++;
33      }
34  }
35
36  size_t RandWriter::orderK() const {
37      return _k;
38  }
39
40  int RandWriter::freq(const std::string& kgram) const {
41      if (kgram.size() != _k) {
42          throw std::invalid_argument("kgram length must equal order k");
43      }
44      auto it = _table.find(kgram);
45      if (it == _table.end()) return 0;
46
47      int total = 0;
48      for (auto const& entry : it->second) {
49          total += entry.second;
50      }
51      return total;
52  }
53
54  int RandWriter::freq(const std::string& kgram, char c) const {
55      if (kgram.size() != _k) {
56          throw std::invalid_argument("kgram length must equal order k");
57      }
58      auto it = _table.find(kgram);
59      if (it == _table.end()) return 0;
60      auto cit = it->second.find(c);
61      if (cit == it->second.end()) return 0;
62      return cit->second;
63  }
64
65  char RandWriter::kRand(const std::string& kgram) {
66      if (kgram.size() != _k) {
67          throw std::invalid_argument("kgram length must equal order k");
68      }
69      auto it = _table.find(kgram);
70      if (it == _table.end()) {
71          throw std::runtime_error("kgram not found in model");
72      }
73
74      int total = freq(kgram);
75      std::uniform_int_distribution<int> dist(1, total);
76      int roll = dist(_rng);
77
78      // Lambda
79      int cumulative = 0;
80      char chosen = it->second.begin()->first;
81      bool found = false;
82      std::for_each(it->second.begin(), it->second.end(),
83          [&](const std::pair<const char, int>& entry) {
84              if (found) return;
85              cumulative += entry.second;
86              if (roll <= cumulative) {
87                  chosen = entry.first;
88                  found = true;
89              }
90          });
91
92      return chosen;
93  }
94
```

```
std::string RandWriter::generate(const std::string& kgram, size_t L) {
    if (kgram.size() != _k) {
        throw std::invalid_argument("kgram length must equal order k");
    }

    std::string result = kgram;
    result.reserve(L);

    if (_k == 0) {
        for (size_t i = 0; i < L; i++) {
            result += kRand("");
        }
        return result;
    }

    while (result.size() < L) {
        std::string current = result.substr(result.size() - _k, _k);
        result += kRand(current);
    }
    return result;
}

std::ostream& operator<<(std::ostream& os, const RandWriter& rw) {
    os << "Order: " << rw._k << "\n";
    os << "Alphabet: " << rw._alphabet << "\n";
    for (auto const& kv : rw._table) {
        os << "\"" << kv.first << "\": ";
        for (auto const& cv : kv.second) {
            os << cv.first << "=" << cv.second << " ";
        }
        os << "\n";
    }
    return os;
}
```

test.cpp

```
// Copyright 2026 Benny
#include <set>
#include <stdexcept>
#include <string>
#define BOOST_TEST_DYN_LINK
#define BOOST_TEST_MODULE RandWriter Tests
#include "RandWriter.hpp"
#include <boost/test/unit_test.hpp>

// kRand test
BOOST_AUTO_TEST_CASE(kRand_test) {
    RandWriter river("mississippi", 2);
    for (int trial = 0; trial < 80; trial++)
        BOOST_REQUIRE_EQUAL(river.kRand("ss"), 'i');
    std::set<char> after_is = {'s', 'p'};
    for (int trial = 0; trial < 80; trial++)
        BOOST_REQUIRE(after_is.count(river.kRand("is")) > 0);
}

// Error Checking
BOOST_AUTO_TEST_CASE(error_checking) {
    RandWriter river("mississippi", 2);
    BOOST_REQUIRE_THROW(river.kRand("m"), std::invalid_argument);
    BOOST_REQUIRE_THROW(river.kRand("mis"), std::invalid_argument);
    BOOST_REQUIRE_THROW(river.kRand("zz"), std::exception);
    BOOST_REQUIRE_THROW(river.freq("i"), std::invalid_argument);
    BOOST_REQUIRE_THROW(river.freq("i", 's'), std::invalid_argument);
    BOOST_REQUIRE_THROW(river.generate("m", 20), std::invalid_argument);
    BOOST_REQUIRE_NO_THROW(river.generate("mi", 20));
```

```
}

// generate() results
BOOST_AUTO_TEST_CASE(generate_results) {
    RandWriter river("mississippi", 2);
    std::set<char> letters = {'m', 'i', 's', 'p'};
    std::string out = river.generate("mi", 11);
    BOOST_REQUIRE_EQUAL(out.size(), 11u);
    BOOST_REQUIRE_EQUAL(out.substr(0, 2), "mi");
    for (char ch : river.generate("mi", 200))
        BOOST_REQUIRE(letters.count(ch) > 0);
    BOOST_REQUIRE_EQUAL(river.generate("mi", 2), "mi");
}

// Letter frequency
BOOST_AUTO_TEST_CASE(letter_frequency) {
    std::string word = "mississippi";
    RandWriter river(word, 2);
    BOOST_REQUIRE(river.freq("ss") > 0);
    BOOST_REQUIRE_EQUAL(river.freq("ss", 'z'), 0);
    BOOST_REQUIRE(river.freq("ss", 'i') > 0);
    BOOST_REQUIRE_EQUAL(river.freq("is"),
        river.freq("is", 's') + river.freq("is", 'p'));

    RandWriter zero(word, 0);
    BOOST_REQUIRE_EQUAL(zero.freq(""), static_cast<int>(word.size()));

    int count_i = 0;
    for (int t = 0; t < 100; t++)
        if (river.kRand("ss") == 'i') count_i++;
    BOOST_REQUIRE_EQUAL(count_i, 100);
}
```

8 PS7: Kronos Log Parsing

8.1 Assignment Overview

Ps7 focuses on analizing system log files from the Kronos Intouch touch-screen time clock device. It reads the log line by line, identifies boot sequences with regular expressions, and produces a report listing each boot. It determines whether each startup completed successfully or not. Then generates a detailed report which are saved into a new rpt file.

8.2 Key Concepts and Design Choices

First design choise was to implement it in python which i'm glad I did. The implementation uses 2 regex expressions. The boot-start pattern captures the timestamp from any line containing (log.c.NNN) server started. The boot-complete pattern captures the timestamp from the line containing oejs.AbstractConnector:Started SelectChannelConnector. State is tracked with a single boolean flag: if a new boot-start line is found while the flag is already set, the previous boot is written as incomplete before the new one is recorded. Timestamps are parsed with Python's datetime.strptime so it's timed correctly.

8.3 What I Learned

The two regex patterns defines a minimal state over the log files. The file is a sequence of (start, complete) pairs, and any unpaired start is an incomplete boot. I also learned to compile regex patterns once before the loop rather than recompiling on every line, which matters on larger files.

8.4 Evidence

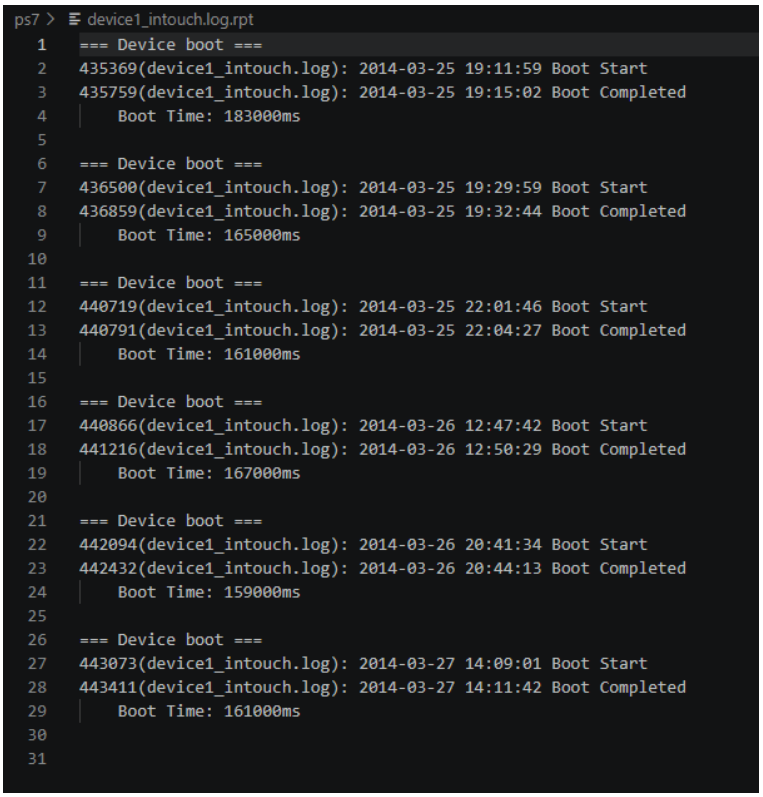


Figure 8: PS7: sample report output showing completed and incomplete boots

8.5 Incomplete Features

No known incomplete features.

8.6 Source Code

ps7.py

```
# Copyright 2026 Benny Chen
import sys
import re
from datetime import datetime

def main():
    if len(sys.argv) < 2:
        print("ps7.py <logfile>")
        sys.exit(1)

    input_path = sys.argv[1]
    filename = input_path.split("/")[-1].split("\\")[-1]
    output_path = input_path + ".rpt"

    # regex boot start and completion
    start_pattern = re.compile(r"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}).*\(log\.c\.166\) server started")
    done_pattern = re.compile(r"(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}).*oejs\.AbstractConnector:Started
         SelectChannelConnector")

    with open(input_path, "r") as f:
        lines = f.readlines()

# track boots
    start_line = None
    start_ts = None

    out = open(output_path, "w")

    for i, line in enumerate(lines):
        m_start = start_pattern.search(line)
        m_done = done_pattern.search(line)

        if m_start:
            if start_line is not None:
                out.write("=== Device boot ===\n")
                out.write(f"{start_line}({filename}): {start_ts} Boot Start\n")
                out.write("**** Incomplete boot **** \n")
                out.write("\n")
            start_line = i + 1
            start_ts = m_start.group(1)

        elif m_done:
            if start_line is not None:
                end_ts = m_done.group(1)
                fmt = "%Y-%m-%d %H:%M:%S"
                t1 = datetime.strptime(start_ts, fmt)
                t2 = datetime.strptime(end_ts, fmt)
                ms = int((t2 - t1).total_seconds() * 1000)
                out.write("=== Device boot ===\n")
                out.write(f"{start_line}({filename}): {start_ts} Boot Start\n")
                out.write(f"{i + 1}({filename}): {end_ts} Boot Completed\n")
                out.write(f"\tBoot Time: {ms}ms \n")
                out.write("\n")
                start_line = None
                start_ts = None

    if start_line is not None:
        out.write("=== Device boot ===\n")
        out.write(f"{start_line}({filename}): {start_ts} Boot Start\n")
```

```
59          out.write("**** Incomplete boot **** \n")
60          out.write("\n")
61
62      out.close()
63
64  if __name__ == "__main__":
65      main()
```